Aprendizaje automático “no neuronal”

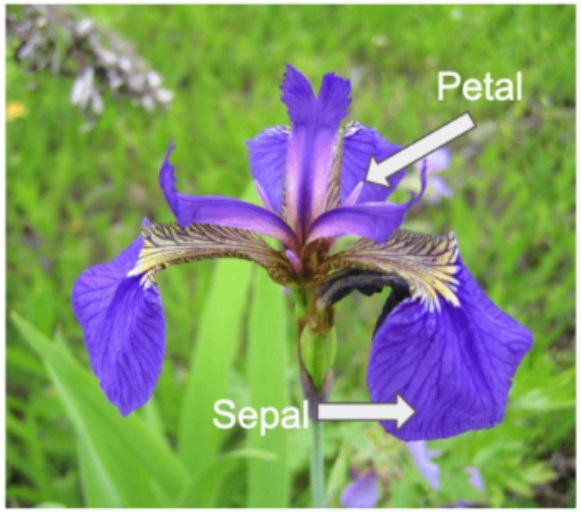
Diego Milone

Inteligencia Computacional
Departamento de Informática

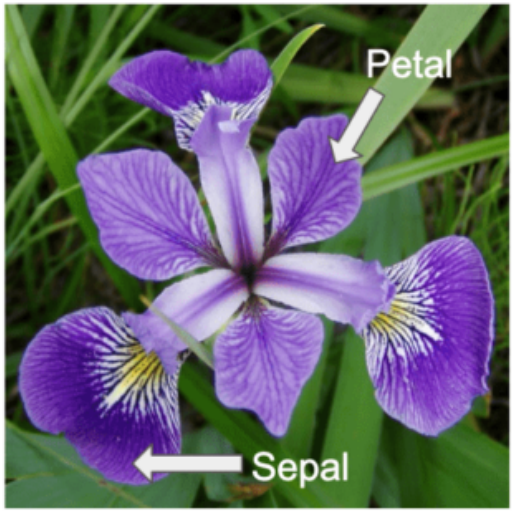
FICH-UNL

Usemos unos datos bien conocidos...

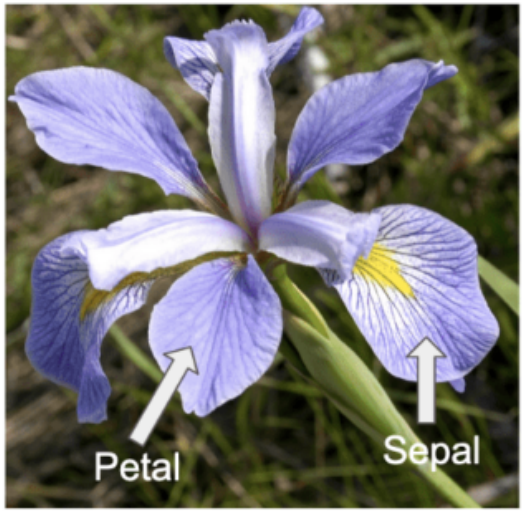
Iris setosa



Iris versicolor



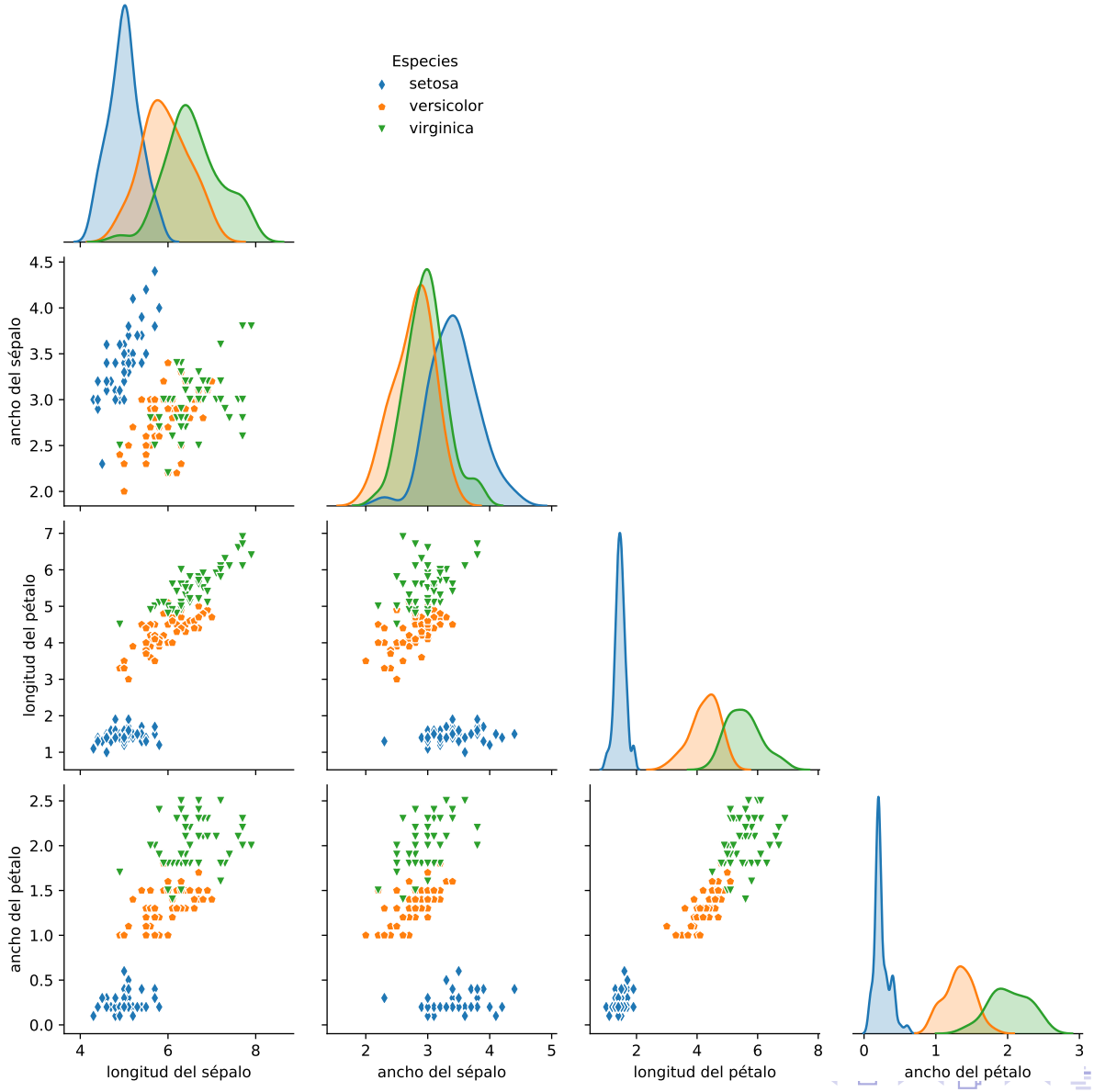
Iris virginica



Mediciones en texto simple

sepal length	sepal width	petal length	petal width	class
5.1	3.5	1.4	0.2	Iris-setosa
4.9	3.0	1.4	0.2	Iris-setosa
4.7	3.2	1.3	0.2	Iris-setosa
4.6	3.1	1.5	0.2	Iris-setosa
5.0	3.6	1.4	0.2	Iris-setosa
5.4	3.9	1.7	0.4	Iris-setosa
4.6	3.4	1.4	0.3	Iris-setosa
5.0	3.4	1.5	0.2	Iris-setosa
4.4	2.9	1.4	0.2	Iris-setosa
4.9	3.1	1.5	0.1	Iris-setosa
5.4	3.7	1.5	0.2	Iris-setosa
4.8	3.4	1.6	0.2	Iris-setosa
4.8	3.0	1.4	0.1	Iris-setosa
4.3	3.0	1.1	0.1	Iris-setosa
5.8	4.0	1.2	0.2	Iris-setosa
5.7	4.4	1.5	0.4	Iris-setosa
5.4	3.9	1.3	0.4	Iris-setosa
5.1	3.5	1.4	0.3	Iris-setosa
5.7	3.8	1.7	0.3	Iris-setosa
5.1	3.8	1.5	0.3	Iris-setosa
5.4	3.4	1.7	0.2	Iris-setosa
5.1	3.7	1.5	0.4	Iris-setosa
4.6	3.6	1.0	0.2	Iris-setosa
5.1	3.3	1.7	0.5	Iris-setosa
4.8	3.4	1.9	0.2	Iris-setosa
5.0	3.0	1.6	0.2	Iris-setosa
5.0	3.4	1.6	0.4	Iris-setosa
5.2	3.5	1.5	0.2	Iris-setosa
5.2	3.4	1.4	0.2	Iris-setosa
4.7	3.2	1.6	0.2	Iris-setosa
4.8	3.1	1.6	0.2	Iris-setosa
5.4	3.4	1.5	0.4	Iris-setosa
5.2	4.1	1.5	0.1	Iris-setosa

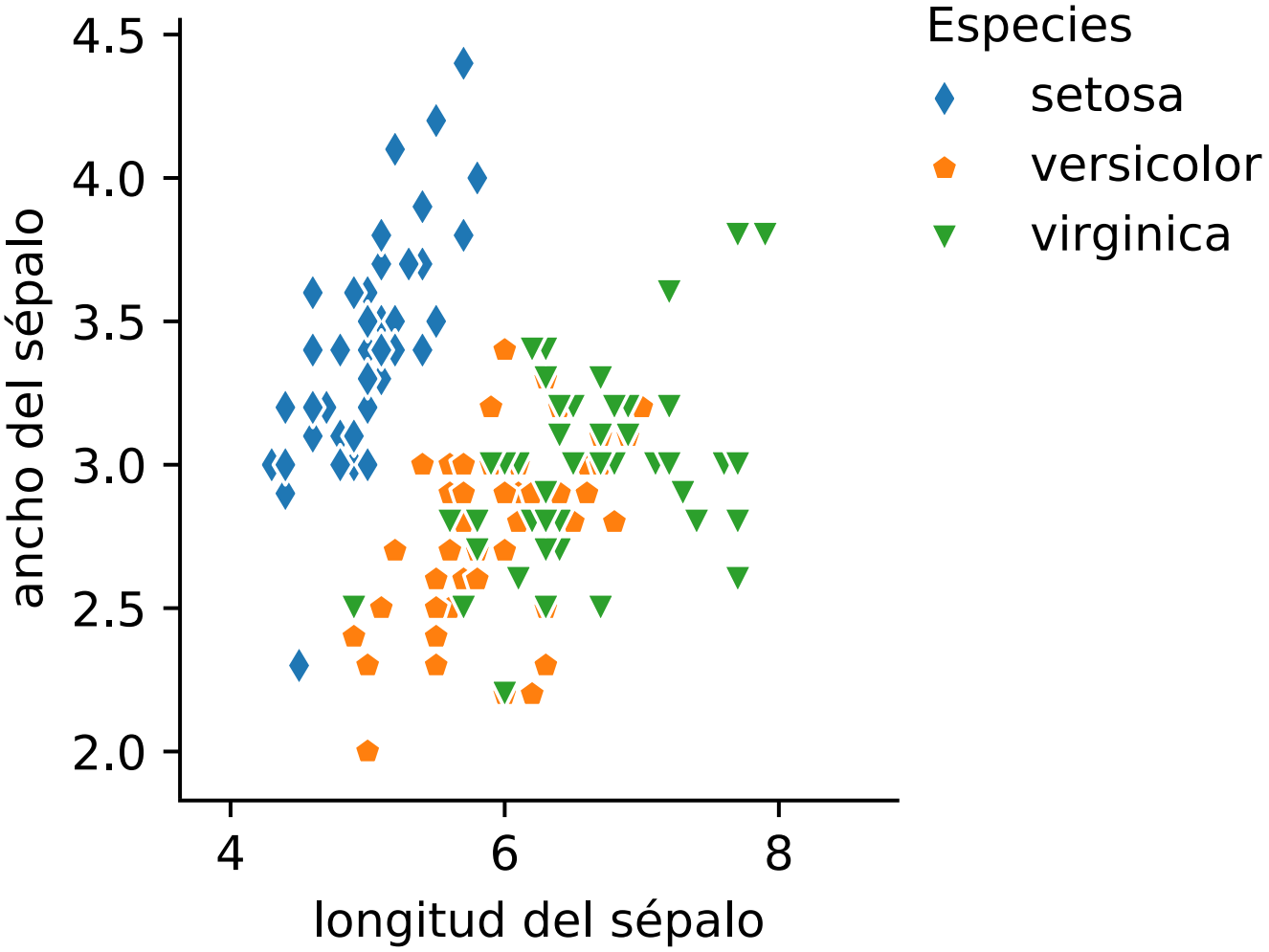
Distribuciones por pares de variables



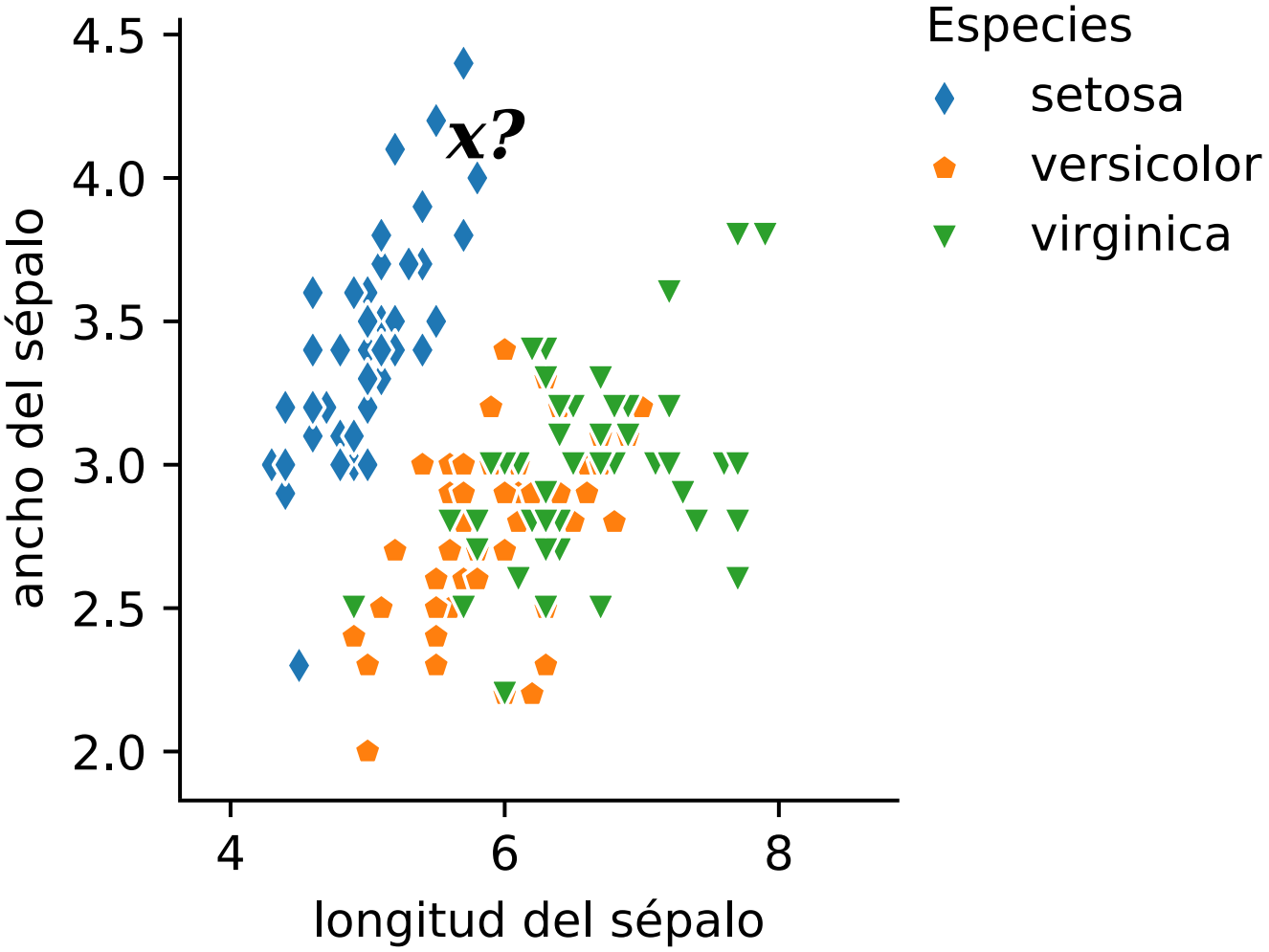
k vecinos más cercanos

(kNN, del inglés k-*Nearest Neighbors*)

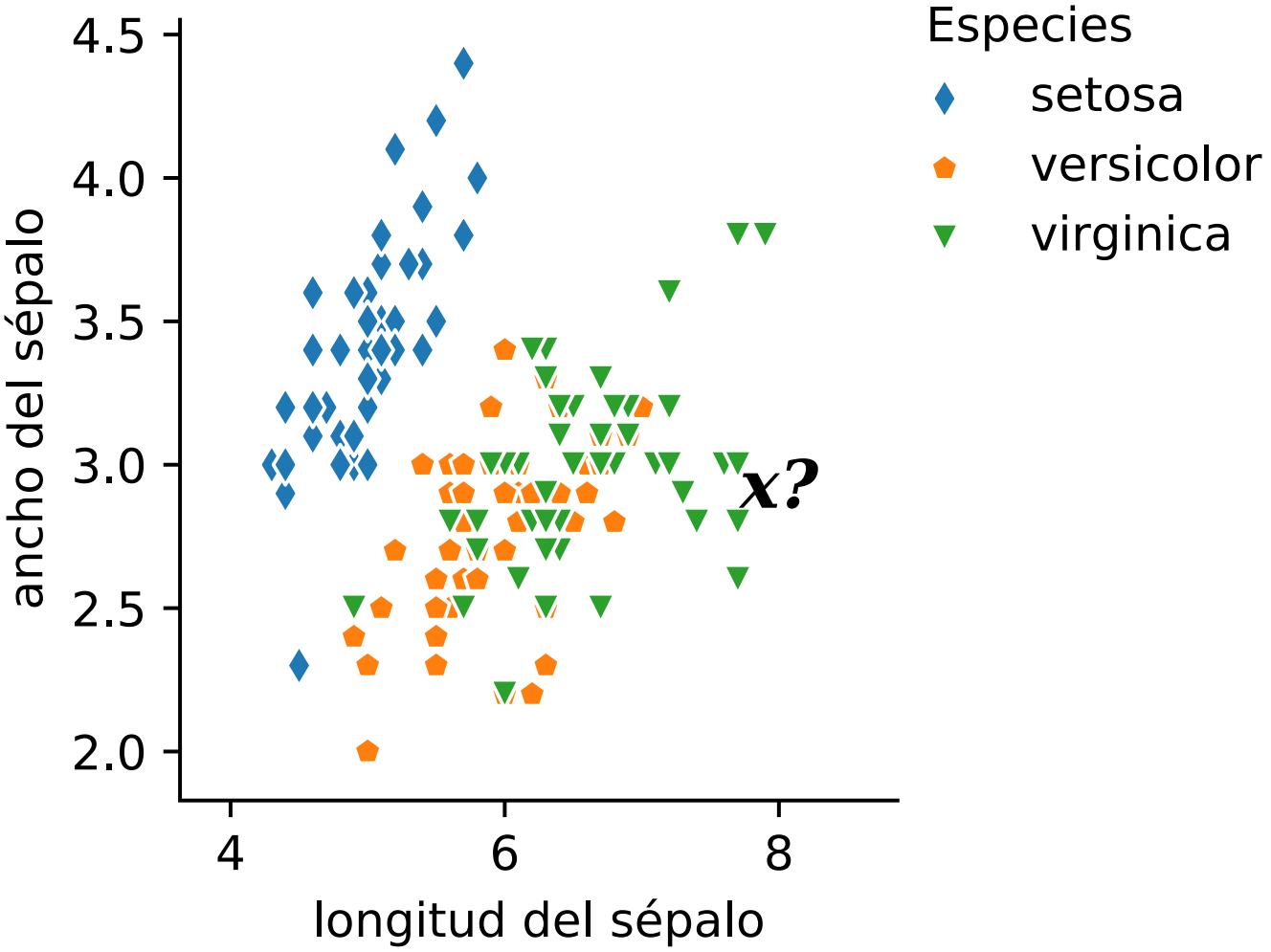
k vecinos más cercanos



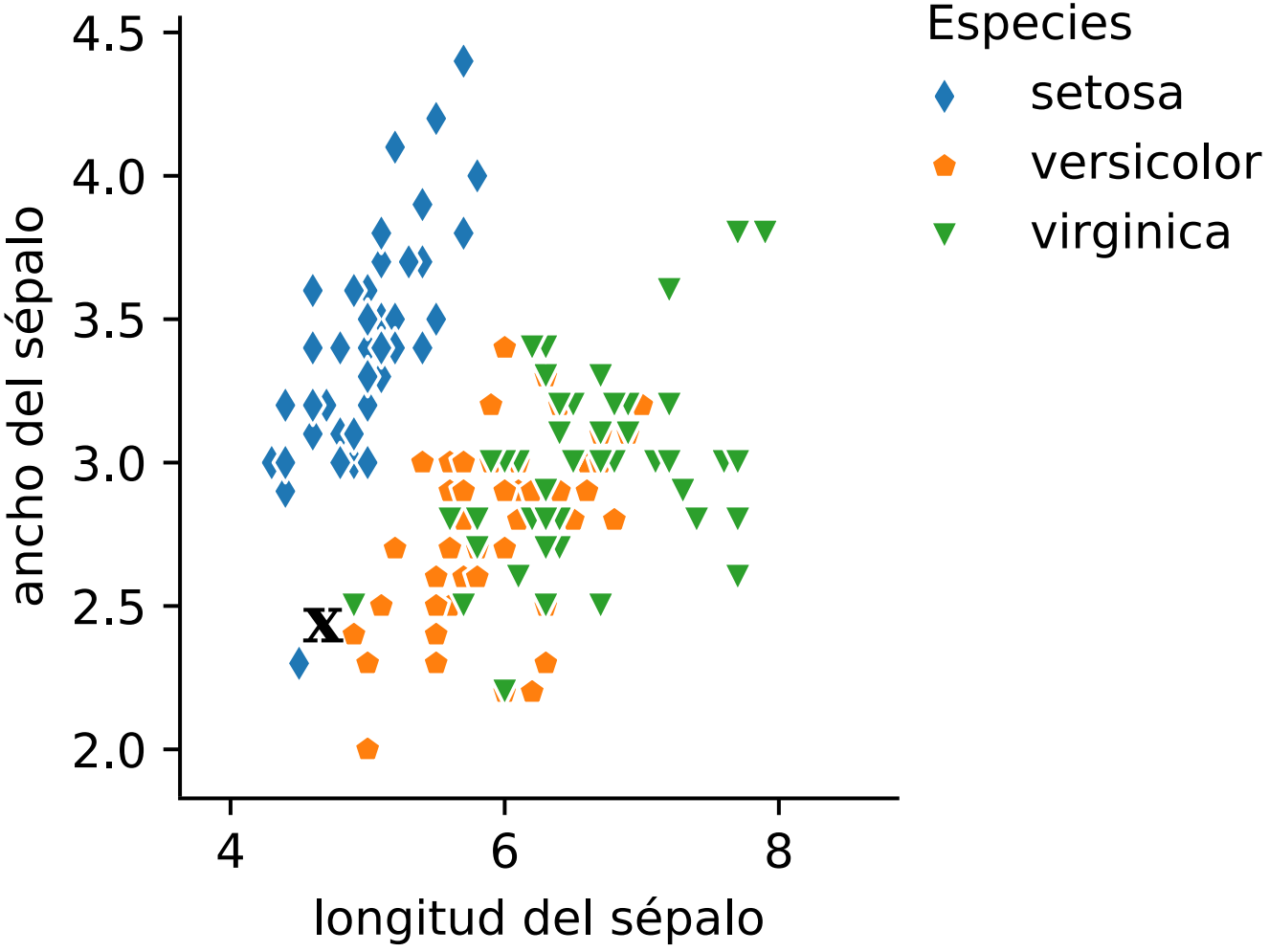
k vecinos más cercanos



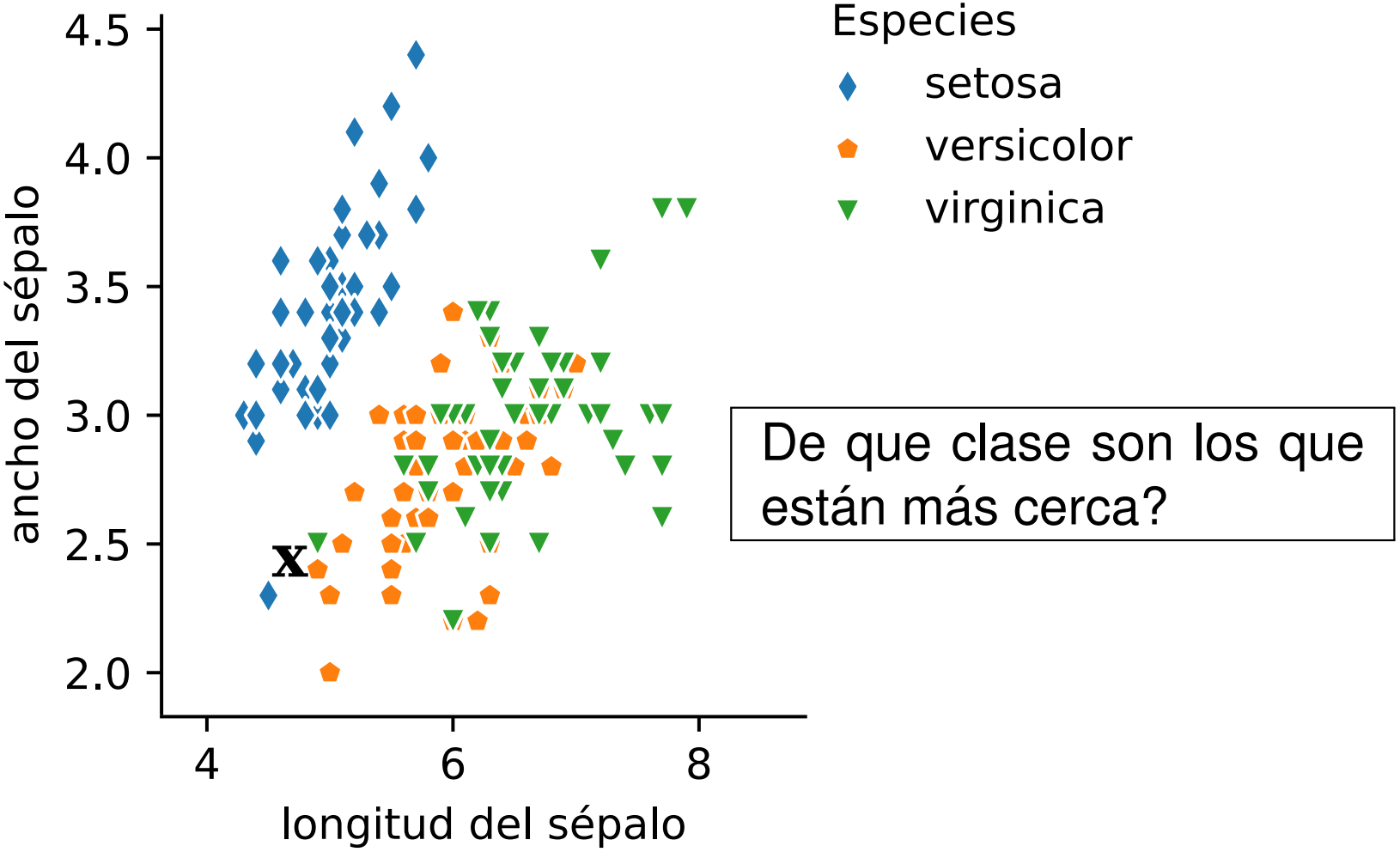
k vecinos más cercanos



k vecinos más cercanos



k vecinos más cercanos



De que clase son los que están más cerca?

k vecinos más cercanos

El algoritmo más simple de aprendizaje automático

k vecinos más cercanos

El algoritmo más simple de aprendizaje automático

- Porque en realidad... no aprende!

k vecinos más cercanos

El algoritmo más simple de aprendizaje automático

- Porque en realidad... no aprende!
- No tiene parámetros que aprender (es no-paramétrico).

k vecinos más cercanos

El algoritmo más simple de aprendizaje automático

- Porque en realidad... no aprende!
- No tiene parámetros que aprender (es no-paramétrico).
- Vuelve a buscar los más cercanos con cada nuevo caso a clasificar (basado en instancias).

k vecinos más cercanos

El algoritmo más simple de aprendizaje automático

- Porque en realidad... no aprende!
- No tiene parámetros que aprender (es no-paramétrico).
- Vuelve a buscar los más cercanos con cada nuevo caso a clasificar (basado en instancias).

Clasificador kNN

Un dato es clasificado por mayoría de votos entre sus vecinos

Clasificador kNN

Supongamos que tenemos:

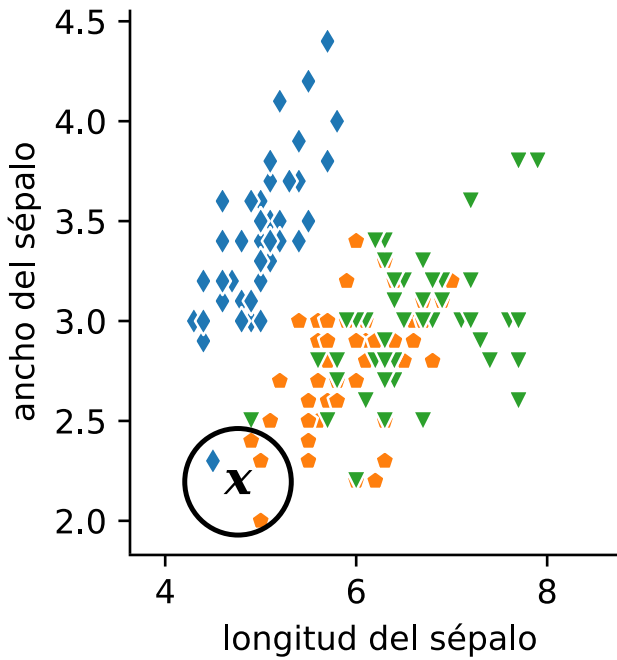
- N_i puntos de la clase ω_i
- N puntos en total $(\sum_i N_i = N)$

Clasificador kNN

Supongamos que tenemos:

- N_i puntos de la clase ω_i
- N puntos en total $(\sum_i N_i = N)$

Para un nuevo $\mathbf{x}$, podemos imaginar una esfera centrada en $\mathbf{x}$



Clasificador kNN

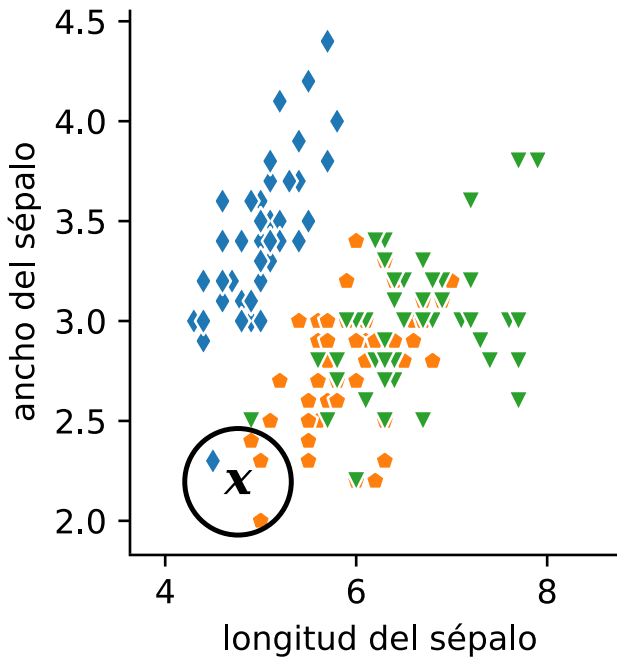
Supongamos que tenemos:

- N_i puntos de la clase ω_i
- N puntos en total $(\sum_i N_i = N)$

Para un nuevo $\mathbf{x}$, podemos imaginar una esfera centrada en $\mathbf{x}$

Supongamos que esta esfera:

- tiene un volumen V
- contiene K puntos en total
- de los cuales K_i son de la clase ω_i



Clasificador kNN

La densidad de la clase i en la esfera sería $\mathrm{p}(\mathbf{x}|\omega_i) = \frac{K_i}{N_i V}$

Clasificador kNN

La densidad de la clase i en la esfera sería $\mathrm{p}(\mathbf{x}|\omega_i) = \frac{K_i}{N_i V}$

Y si no condicionamos a la clase tendríamos $\mathrm{p}(\mathbf{x}) = \frac{K}{NV}$

Clasificador kNN

La densidad de la clase i en la esfera sería $\mathrm{p}(\mathbf{x}|\omega_i) = \frac{K_i}{N_i V}$

Y si no condicionamos a la clase tendríamos $\mathrm{p}(\mathbf{x}) = \dfrac{K}{NV}$

Lo que podríamos esperar para cada clase sería $\mathrm{p}(\omega_i) = \dfrac{N_i}{N}$

Clasificador kNN

La densidad de la clase i en la esfera sería $\mathrm{p}(\mathbf{x}|\omega_i) = \frac{K_i}{N_i V}$

Y si no condicionamos a la clase tendríamos $\mathrm{p}(\mathbf{x}) = \dfrac{K}{NV}$

Lo que podríamos esperar para cada clase sería $\mathrm{p}(\omega_i) = \dfrac{N_i}{N}$

Y si con esto usamos el teorema de Bayes

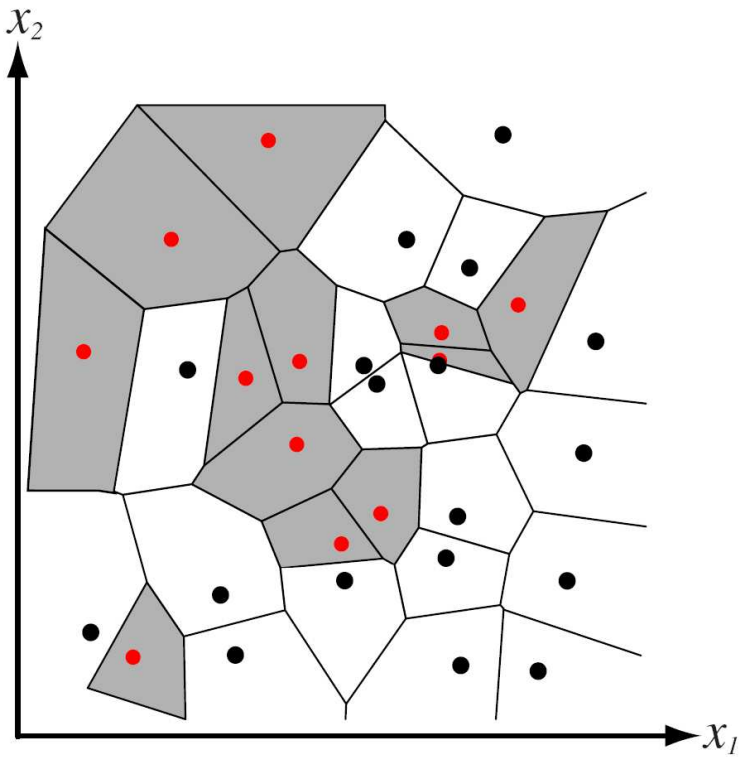
$$\mathrm{p}(\omega_i|\mathbf{x}) = \frac{\mathrm{p}(\mathbf{x}|\omega_i)\mathrm{p}(\omega_i)}{\mathrm{p}(\mathbf{x})} = \frac{\frac{K_i}{N_i V}\frac{N_i}{N}}{\frac{K}{NV}} = \frac{K_i}{K}$$

Así, el clasificador kNN asigna a $\mathbf{x}$ la clase ω_i que tiene la máxima cantidad de patrones K_i en la esfera.

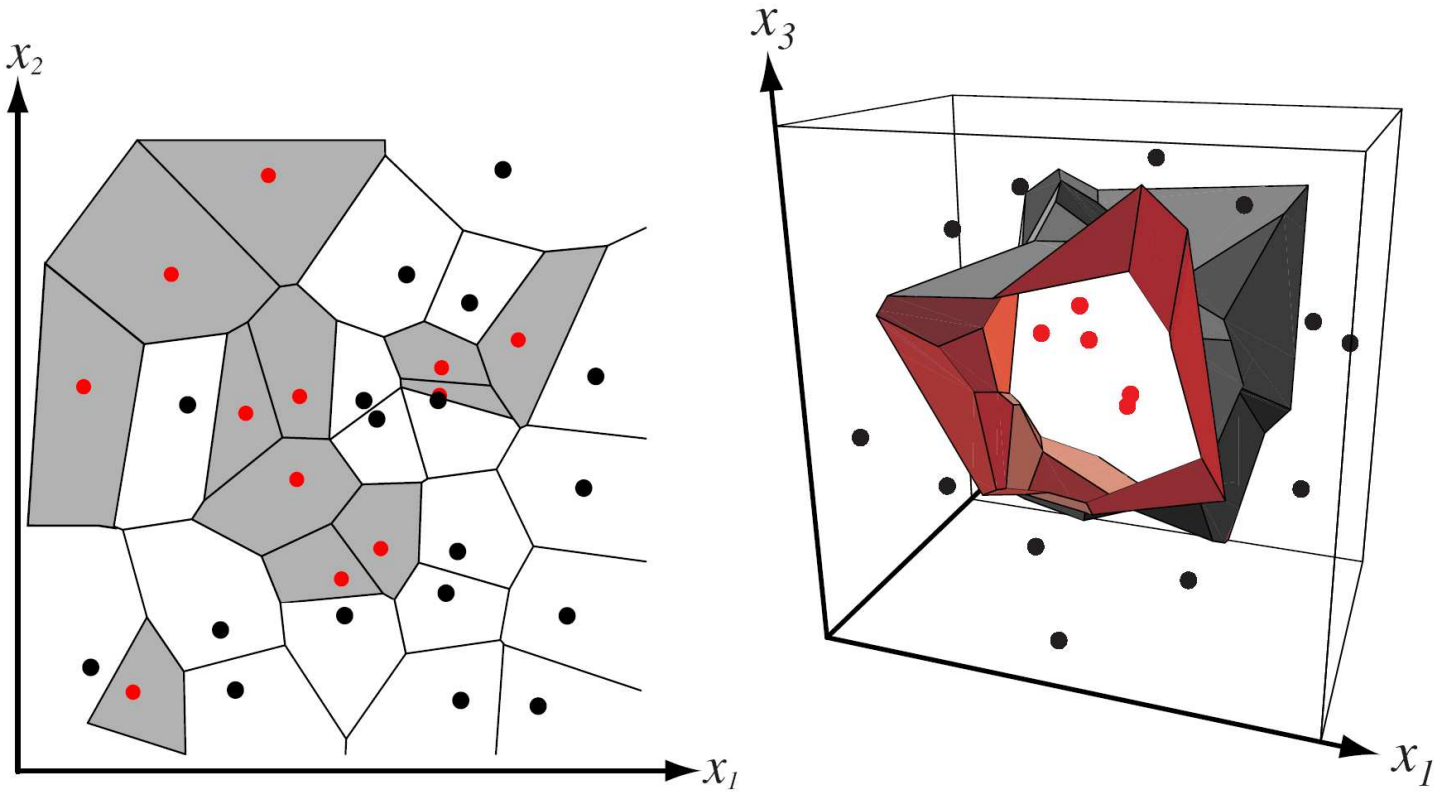
kNN con k vecinos

(en lugar de una esfera de radio fijo)

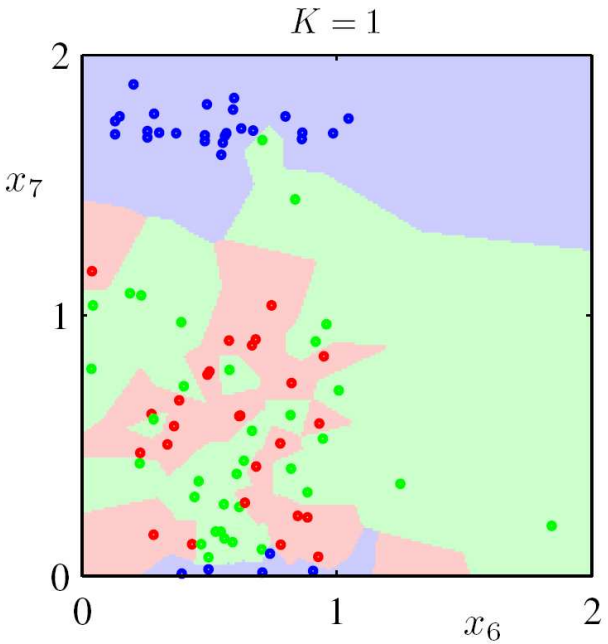
kNN con $k = 1$ vecinos



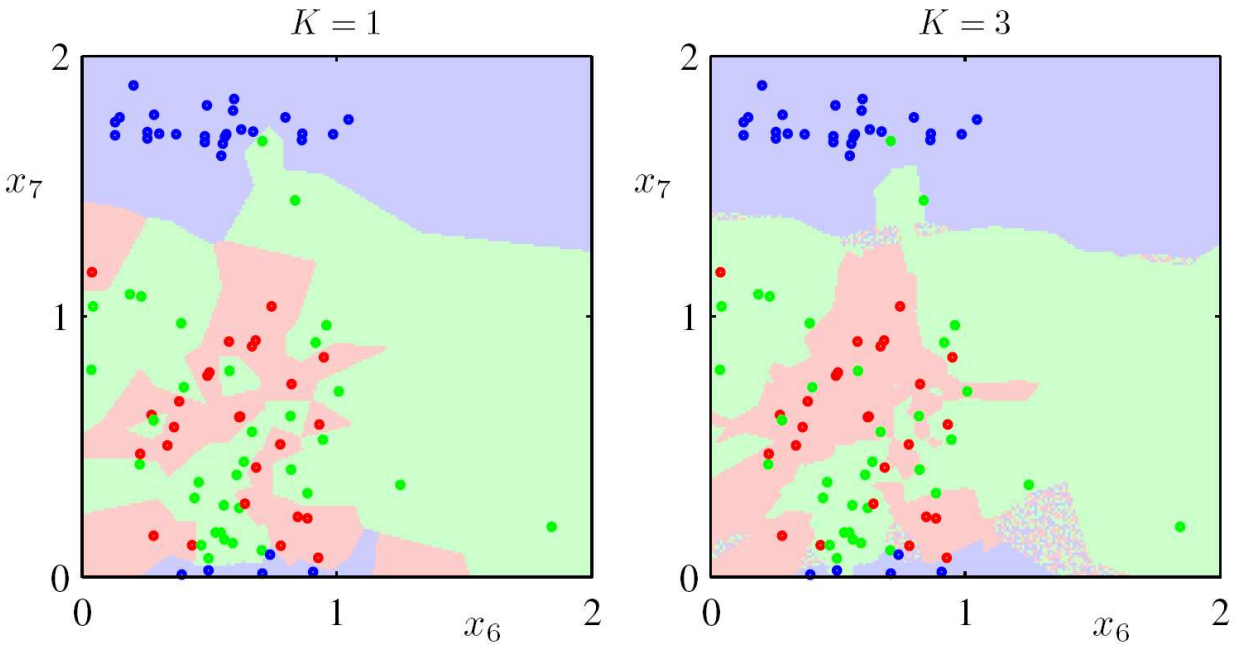
kNN con $k = 1$



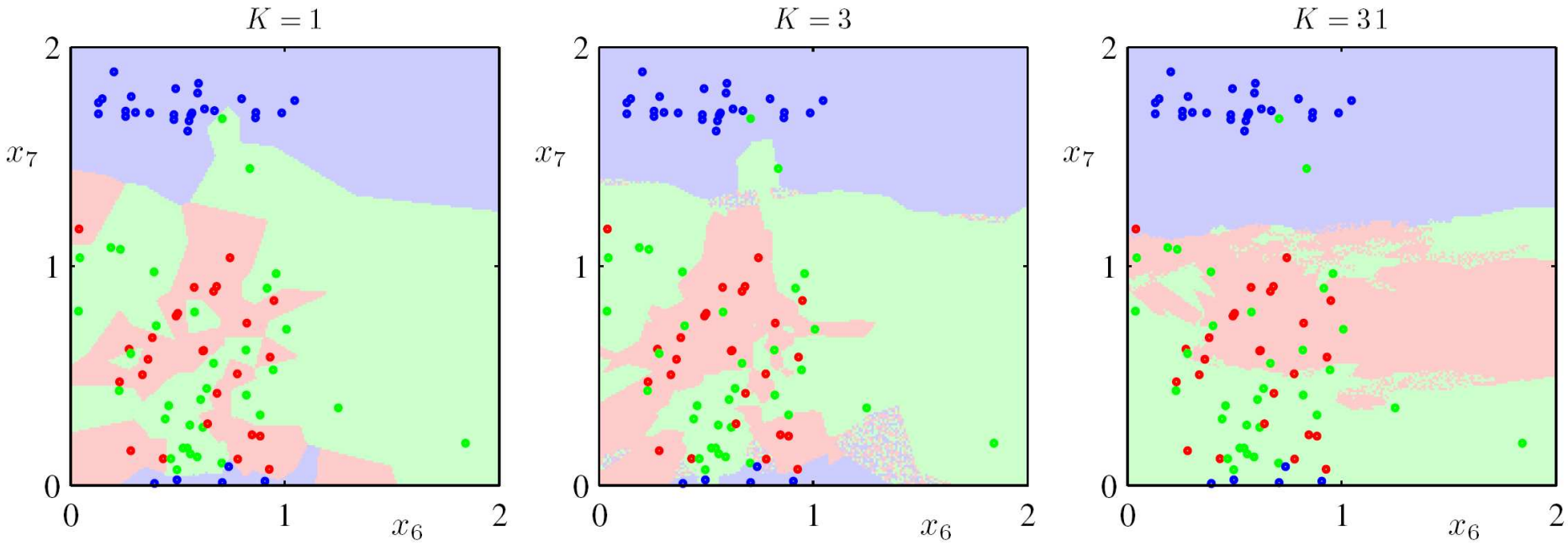
kNN: fronteras de decisión



kNN: fronteras de decisión



kNN: fronteras de decisión



k vecinos más cercanos

Ventajas:

- Muy simple: no hay una regla de aprendizaje, de hecho... no hay aprendizaje!

Desventajas:

k vecinos más cercanos

Ventajas:

- Muy simple: no hay una regla de aprendizaje, de hecho... no hay aprendizaje!
- No hay parámetros entrenables

Desventajas:

k vecinos más cercanos

Ventajas:

- Muy simple: no hay una regla de aprendizaje, de hecho... no hay aprendizaje!
- No hay parámetros entrenables
- Es bastante efectivo si el espacio está bien cubierto por los datos de entrenamiento.

Desventajas:

k vecinos más cercanos

Ventajas:

- Muy simple: no hay una regla de aprendizaje, de hecho... no hay aprendizaje!
- No hay parámetros entrenables
- Es bastante efectivo si el espacio está bien cubierto por los datos de entrenamiento.
- Puede hacer bastante robusto al ruido en los datos (con el k apropiado).

Desventajas:

k vecinos más cercanos

Ventajas:

- Muy simple: no hay una regla de aprendizaje, de hecho... no hay aprendizaje!
- No hay parámetros entrenables
- Es bastante efectivo si el espacio está bien cubierto por los datos de entrenamiento.
- Puede hacer bastante robusto al ruido en los datos (con el k apropiado).

Desventajas:

- Hay que determinar el valor óptimo de k (ahh... entonces sí había un parámetro??)

k vecinos más cercanos

Ventajas:

- Muy simple: no hay una regla de aprendizaje, de hecho... no hay aprendizaje!
- No hay parámetros entrenables
- Es bastante efectivo si el espacio está bien cubierto por los datos de entrenamiento.
- Puede hacer bastante robusto al ruido en los datos (con el k apropiado).

Desventajas:

- Hay que determinar el valor óptimo de k
- No es tan claro qué distancia usar en cada caso.

k vecinos más cercanos

Ventajas:

- Muy simple: no hay una regla de aprendizaje, de hecho... no hay aprendizaje!
- No hay parámetros entrenables
- Es bastante efectivo si el espacio está bien cubierto por los datos de entrenamiento.
- Puede hacer bastante robusto al ruido en los datos (con el k apropiado).

Desventajas:

- Hay que determinar el valor óptimo de k
- No es tan claro qué distancia usar en cada caso.
- Para cada dato de prueba requiere calcular la distancia a todos los datos de entrenamiento.

k vecinos más cercanos

Ventajas:

- Muy simple: no hay una regla de aprendizaje, de hecho... no hay aprendizaje!
- No hay parámetros entrenables
- Es bastante efectivo si el espacio está bien cubierto por los datos de entrenamiento.
- Puede hacer bastante robusto al ruido en los datos (con el k apropiado).

Desventajas:

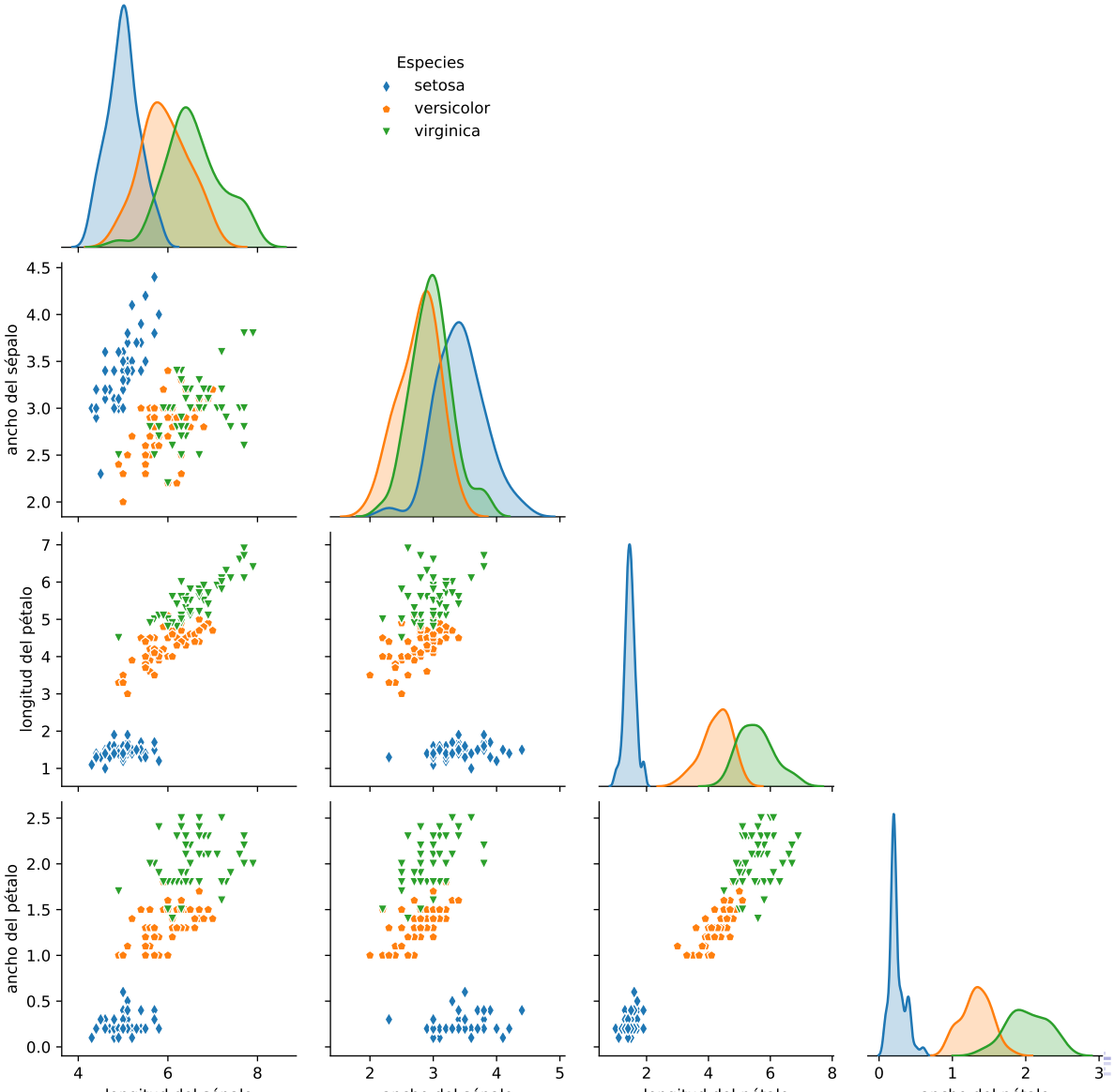
- Hay que determinar el valor óptimo de k
- No es tan claro qué distancia usar en cada caso.
- Para cada dato de prueba requiere calcular la distancia a todos los datos de entrenamiento.
- Sensible a outliers y clases desbalanceadas.

Árboles de decisión

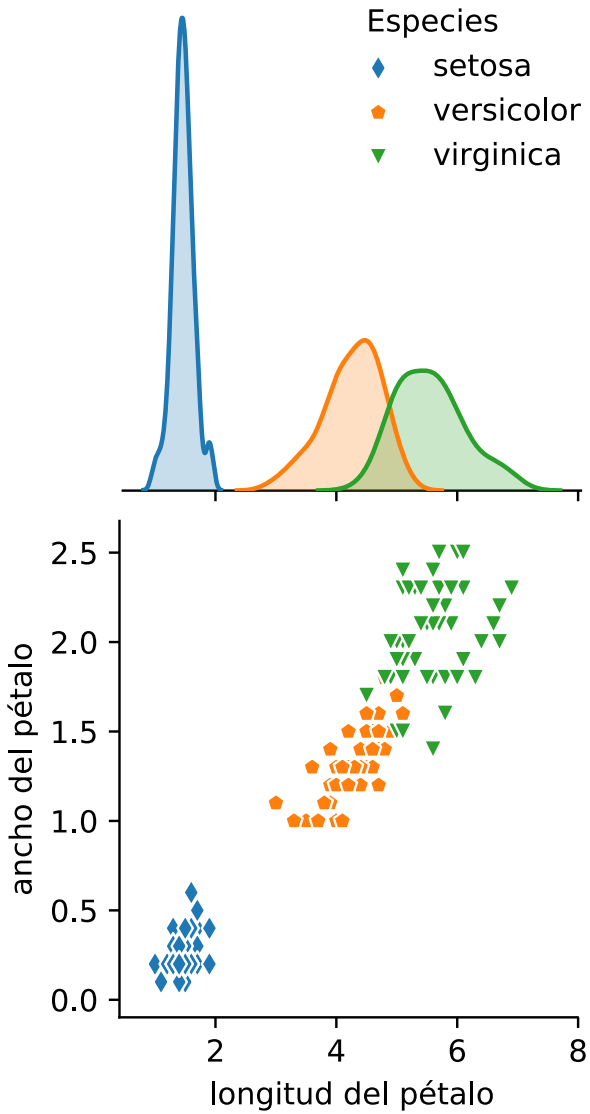
(DT, del inglés *Decision Trees*)

Árboles de decisión

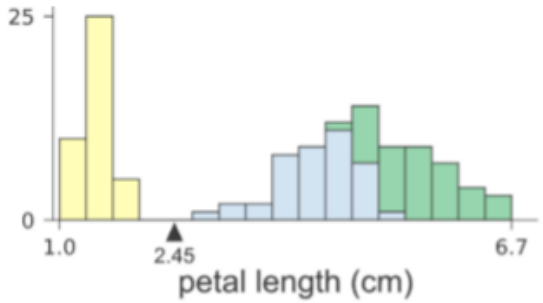
(DT, del inglés *Decision Trees*)



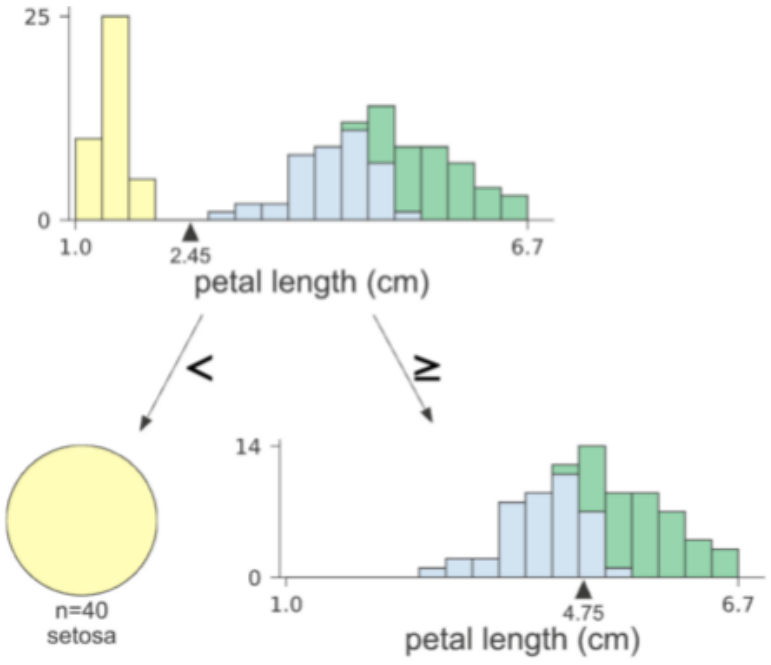
Árboles de decisión



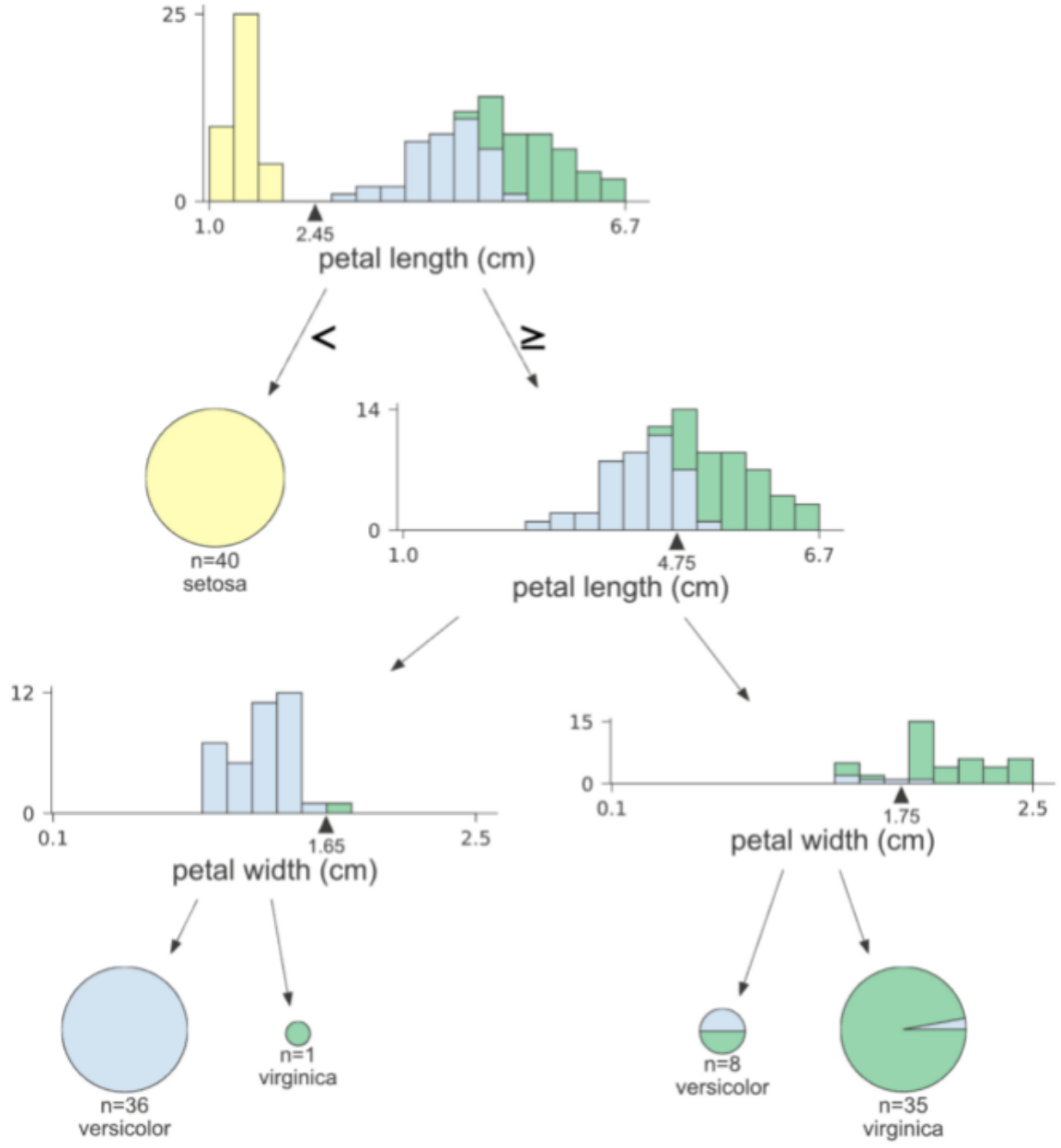
Árboles de decisión



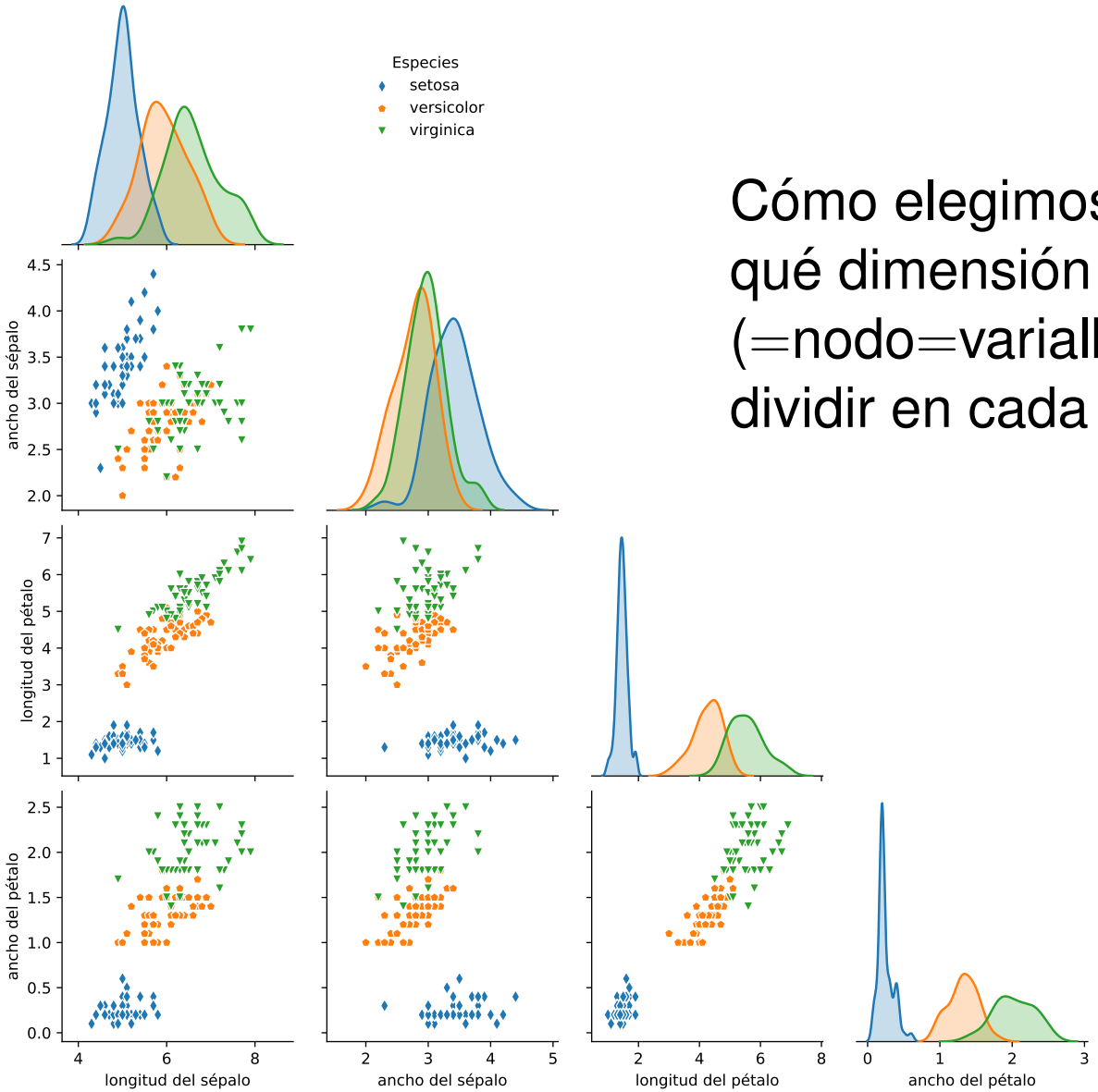
Árboles de decisión



Árboles de decisión



Árboles de decisión



Cómo elegimos por qué dimensión (=nodo=varialbe) dividir en cada caso?

Árboles de decisión: medidas de impureza

Impureza de clasificación en el nodo n:

$$\mathcal{I}_n^C = 1 - \max_i \mathsf{P}(\mathbf{x} \in w_i | n)$$

Árboles de decisión: medidas de impureza

Impureza de clasificación en el nodo n:

$$\mathcal{I}_n^C = 1 - \max_i \mathsf{P}(\mathbf{x} \in w_i | n)$$

Impureza de Gini:

$$\begin{aligned} \mathcal{I}_n^G &= \sum_{i \neq j} \mathsf{P}(\mathbf{x} \in w_i | n) \mathsf{P}(\mathbf{x} \in w_j | n) \\ &= 1 - \sum_i \mathsf{P}^2(\mathbf{x} \in w_i | n) \end{aligned}$$

Árboles de decisión: medidas de impureza

Impureza de clasificación en el nodo n:

$$\mathcal{I}_n^C = 1 - \max_i \mathsf{P}(\mathbf{x} \in w_i | n)$$

Impureza de Gini:

$$\begin{aligned} \mathcal{I}_n^G &= \sum_{i \neq j} \mathsf{P}(\mathbf{x} \in w_i | n) \mathsf{P}(\mathbf{x} \in w_j | n) \\ &= 1 - \sum_i \mathsf{P}^2(\mathbf{x} \in w_i | n) \end{aligned}$$

Impureza de Entropía:

$$\mathcal{I}_n^H = -\sum_i \mathsf{P}(\mathbf{x} \in w_i | n) \log \mathsf{P}(\mathbf{x} \in w_i | n)$$

Árboles de decisión: medidas de impureza

Impureza de clasificación en el nodo n:

$$\mathcal{I}_n^C = 1 - \max_i \mathsf{P}(\mathbf{x} \in w_i | n)$$

Impureza de Gini:

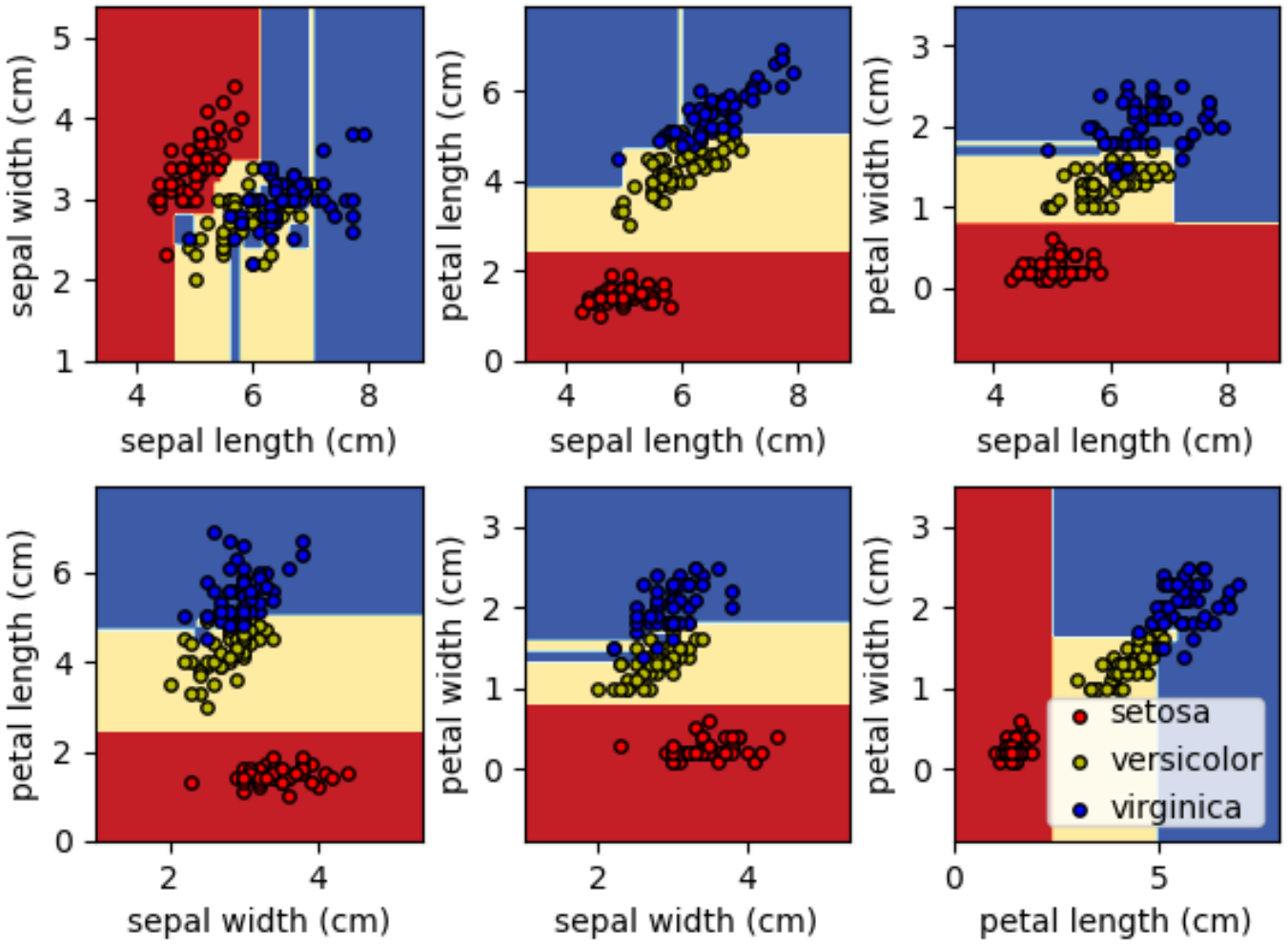
$$\begin{aligned} \mathcal{I}_n^G &= \sum_{i \neq j} \mathsf{P}(\mathbf{x} \in w_i | n) \mathsf{P}(\mathbf{x} \in w_j | n) \\ &= 1 - \sum_i \mathsf{P}^2(\mathbf{x} \in w_i | n) \end{aligned}$$

Impureza de Entropía:

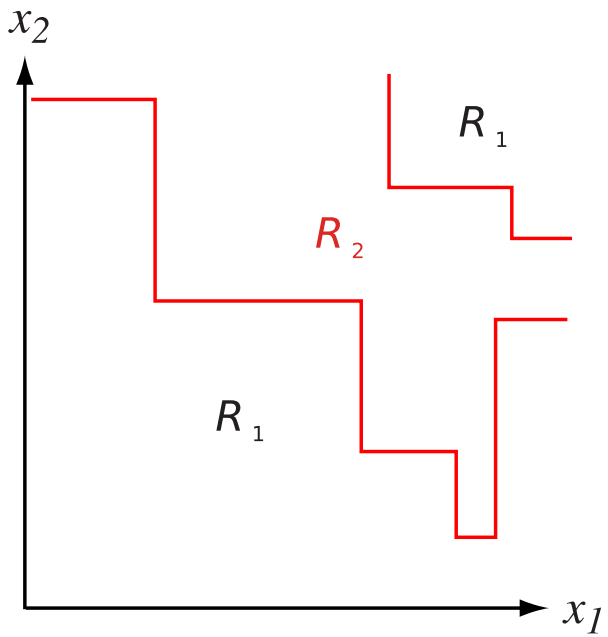
$$\mathcal{I}_n^H = -\sum_i \mathsf{P}(\mathbf{x} \in w_i | n) \log \mathsf{P}(\mathbf{x} \in w_i | n)$$

Ganancia de información: $\mathcal{G} = \mathcal{I}_p - \sum_h \frac{N_h}{N} \mathcal{I}_h$

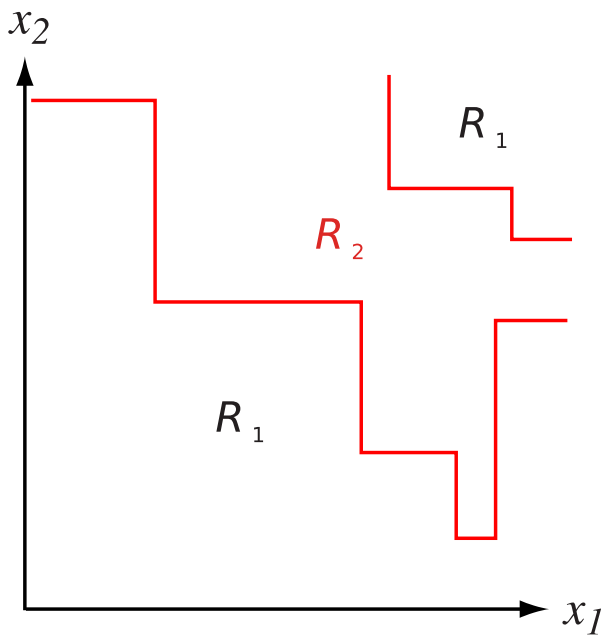
Árboles de decisión: fronteras de decisión

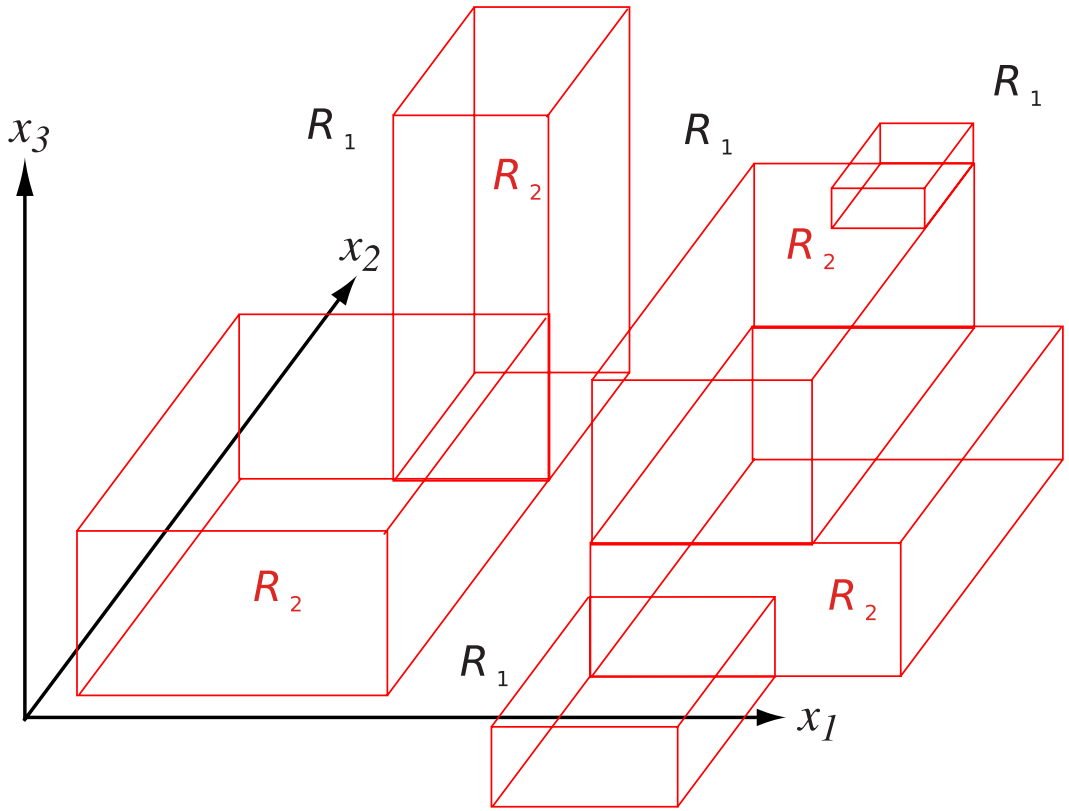


Árboles de decisión: fronteras de decisión



Árboles de decisión: fronteras de decisión





Árboles de decisión

Ventajas:

- Proveen una explicación simple de la decisión con reglas IF-THEN.

Desventajas:

Árboles de decisión

Ventajas:

- Proveen una explicación simple de la decisión con reglas IF-THEN.
- Tratan de forma natural tanto datos numéricos como categóricos (y a la vez) sin requerir transformaciones.

Desventajas:

Árboles de decisión

Ventajas:

- Proveen una explicación simple de la decisión con reglas IF-THEN.
- Tratan de forma natural tanto datos numéricos como categóricos (y a la vez) sin requerir transformaciones.
- Tienen bajo costo computacional tanto para en entrenamiento como para su uso en producción.

Desventajas:

Árboles de decisión

Ventajas:

- Proveen una explicación simple de la decisión con reglas IF-THEN.
- Tratan de forma natural tanto datos numéricos como categóricos (y a la vez) sin requerir transformaciones.
- Tienen bajo costo computacional tanto para en entrenamiento como para su uso en producción.
- No requieren grandes cantidades de datos para entrenar.

Desventajas:

Árboles de decisión

Ventajas:

- Proveen una explicación simple de la decisión con reglas IF-THEN.
- Tratan de forma natural tanto datos numéricos como categóricos (y a la vez) sin requerir transformaciones.
- Tienen bajo costo computacional tanto para en entrenamiento como para su uso en producción.
- No requieren grandes cantidades de datos para entrenar.
- Bastante robustos a los outliers.

Desventajas:

Árboles de decisión

Ventajas:

- Proveen una explicación simple de la decisión con reglas IF-THEN.
- Tratan de forma natural tanto datos numéricos como categóricos (y a la vez) sin requerir transformaciones.
- Tienen bajo costo computacional tanto para en entrenamiento como para su uso en producción.
- No requieren grandes cantidades de datos para entrenar.
- Bastante robustos a los outliers.

Desventajas:

- No son robustos a ruido (pequeños cambios) en los datos.

Árboles de decisión

Ventajas:

- Proveen una explicación simple de la decisión con reglas IF-THEN.
- Tratan de forma natural tanto datos numéricos como categóricos (y a la vez) sin requerir transformaciones.
- Tienen bajo costo computacional tanto para en entrenamiento como para su uso en producción.
- No requieren grandes cantidades de datos para entrenar.
- Bastante robustos a los outliers.

Desventajas:

- No son robustos a ruido (pequeños cambios) en los datos.
- Fronteras de decisión ortogonales a las dimensiones de entrada.

Árboles de decisión

Ventajas:

- Proveen una explicación simple de la decisión con reglas IF-THEN.
- Tratan de forma natural tanto datos numéricos como categóricos (y a la vez) sin requerir transformaciones.
- Tienen bajo costo computacional tanto para en entrenamiento como para su uso en producción.
- No requieren grandes cantidades de datos para entrenar.
- Bastante robustos a los outliers.

Desventajas:

- No son robustos a ruido (pequeños cambios) en los datos.
- Fronteras de decisión ortogonales a las dimensiones de entrada.
- No escalan bien con la complejidad de los problemas a resolver.

Árboles de decisión

Ventajas:

- Proveen una explicación simple de la decisión con reglas IF-THEN.
- Tratan de forma natural tanto datos numéricos como categóricos (y a la vez) sin requerir transformaciones.
- Tienen bajo costo computacional tanto para en entrenamiento como para su uso en producción.
- No requieren grandes cantidades de datos para entrenar.
- Bastante robustos a los outliers.

Desventajas:

- No son robustos a ruido (pequeños cambios) en los datos.
- Fronteras de decisión ortogonales a las dimensiones de entrada.
- No escalan bien con la complejidad de los problemas a resolver.
- Se sobreajustan con facilidad.

Clasificadores Bayesianos Ingenuos

(NB, del inglés *Naive Bayes*)

Naive Bayes

Construyamos un clasificador usando el teorema de Bayes:

Naive Bayes

Construyamos un clasificador usando el teorema de Bayes:

$$\mathrm{P}(w|\mathbf{x}) = \frac{\mathrm{P}(\mathbf{x}|w)\mathrm{P}(w)}{\mathrm{P}(\mathbf{x})}$$

Naive Bayes

Construyamos un clasificador usando el teorema de Bayes:

$$\mathrm{P}(w|\mathbf{x}) = \frac{\mathrm{P}(\mathbf{x}|w)\mathrm{P}(w)}{\mathrm{P}(\mathbf{x})}$$

...y supongamos que las dimensiones o características son condicionalmente independientes (esta es la parte más ingenua!)

$$\mathrm{P}(x_i|w, x_j) = \mathrm{P}(x_i|w) \quad \forall i \neq j.$$

Naive Bayes

Según la definición de probabilidad condicional podemos hacer

$$\mathsf{P}(x_1, x_2, .., x_n|w)\mathsf{P}(w) = \mathsf{P}(w, x_1, x_2, \ldots, x_n),$$

Naive Bayes

Según la definición de probabilidad condicional podemos hacer

$$\mathsf{P}(x_1, x_2, .., x_n|w)\mathsf{P}(w) = \mathsf{P}(w, x_1, x_2, \ldots, x_n),$$

y usando en cadena esta misma definición

$$\begin{aligned}
\mathsf{P}(w, x_1, x_2, \ldots, x_n) &= \mathsf{P}(x_1, x_2, \ldots, x_n|w)\mathsf{P}(w) \\
&= \mathsf{P}(x_2, x_3, \ldots, x_n|w, x_1)\mathsf{P}(w)\mathsf{P}(x_1|w) \\
&= \mathsf{P}(x_3, x_4, \ldots, x_n|w, x_1, x_2)\mathsf{P}(w)\mathsf{P}(x_1|w)\mathsf{P}(x_2|w, x_1) \\
&= \ldots
\end{aligned}$$

Naive Bayes

Según la definición de probabilidad condicional podemos hacer

$$\mathsf{P}(x_1, x_2, .., x_n|w)\mathsf{P}(w) = \mathsf{P}(w, x_1, x_2, \ldots, x_n),$$

y usando en cadena esta misma definición

$$\begin{aligned}
\mathsf{P}(w, x_1, x_2, \ldots, x_n) &= \mathsf{P}(x_1, x_2, \ldots, x_n|w)\mathsf{P}(w) \\
&= \mathsf{P}(x_2, x_3, \ldots, x_n|w, x_1)\mathsf{P}(w)\mathsf{P}(x_1|w) \\
&= \mathsf{P}(x_3, x_4, \ldots, x_n|w, x_1, x_2)\mathsf{P}(w)\mathsf{P}(x_1|w)\mathsf{P}(x_2|w, x_1) \\
&= \ldots
\end{aligned}$$

pero con la suposición *ingenua* de que $\mathsf{P}(x_i|w, x_j) = \mathsf{P}(x_i|w)$

Naive Bayes

Según la definición de probabilidad condicional podemos hacer

$$\mathsf{P}(x_1, x_2, .., x_n|w)\mathsf{P}(w) = \mathsf{P}(w, x_1, x_2, \ldots, x_n),$$

y usando en cadena esta misma definición

$$\begin{aligned}
\mathsf{P}(w, x_1, x_2, \ldots, x_n) &= \mathsf{P}(x_1, x_2, \ldots, x_n|w)\mathsf{P}(w) \\
&= \mathsf{P}(x_2, x_3, \ldots, x_n|w, x_1)\mathsf{P}(w)\mathsf{P}(x_1|w) \\
&= \mathsf{P}(x_3, x_4, \ldots, x_n|w, x_1, x_2)\mathsf{P}(w)\mathsf{P}(x_1|w)\mathsf{P}(x_2|w, x_1) \\
&= \ldots
\end{aligned}$$

pero con la suposición *ingenua* de que $\mathsf{P}(x_i|w, x_j) = \mathsf{P}(x_i|w)$

$$\begin{aligned}
\mathsf{P}(w, x_1, x_2, .., x_n) &= \mathsf{P}(w)\mathsf{P}(x_1|w)\mathsf{P}(x_2|w)\ldots\mathsf{P}(x_n|w) \\
&= \mathsf{P}(w)\prod_{i=1}^{n}\mathsf{P}(x_i|w)
\end{aligned}$$

Naive Bayes classification

De esta forma nos queda

$$\mathrm{P}(w|\mathbf{x}) = \frac{\mathrm{P}(\mathbf{x}|w)\mathrm{P}(w)}{\mathrm{P}(\mathbf{x})} = \frac{\mathrm{P}(w)\prod_{i=1}^{n}\mathrm{P}(x_i|w)}{\mathrm{P}(\mathbf{x})}$$

Y considerando la misma probabilidad $\mathrm{P}(w)$ para cada clase, y que $\mathrm{P}(\mathbf{x})$ no depende de w, la clase más probable se puede obtener mediante

$$\hat{w}(\mathbf{x}) = \arg\max_{w} \prod_{i=1}^{n} \mathrm{P}(x_i|w)$$

NB: solo nos queda calcular las $P(x_i|w)$

NB: solo nos queda calcular las $\mathrm{P}(x_i|w)$

NB Gaussiano $\mathcal{N}(\mu, \sigma^2)$

$$\mathrm{P}(x_i|w) = \frac{1}{\sigma_w\sqrt{2\pi}} e^{-\frac{(x_i-\mu_w)^2}{2\sigma_w^2}}$$

NB: pero no es tan *ingenuo*!

Ventajas:

- Puede usarse directamente en mezclas de datos continuos y categóricos.

Desventajas:

NB: pero no es tan *ingenuo*!

Ventajas:

- Puede usarse directamente en mezclas de datos continuos y categóricos.
- En varios conjuntos reales funciona mejor que los árboles de decisión y otros métodos simples.

Desventajas:

NB: pero no es tan *ingenuo*!

Ventajas:

- Puede usarse directamente en mezclas de datos continuos y categóricos.
- En varios conjuntos reales funciona mejor que los árboles de decisión y otros métodos simples.
- El tiempo de entrenamiento crece linealmente con la cantidad de dimensiones.

Desventajas:

NB: pero no es tan *ingenuo*!

Ventajas:

- Puede usarse directamente en mezclas de datos continuos y categóricos.
- En varios conjuntos reales funciona mejor que los árboles de decisión y otros métodos simples.
- El tiempo de entrenamiento crece linealmente con la cantidad de dimensiones.
- Rápido y simple de implementar.

Desventajas:

NB: pero no es tan *ingenuo*!

Ventajas:

- Puede usarse directamente en mezclas de datos continuos y categóricos.
- En varios conjuntos reales funciona mejor que los árboles de decisión y otros métodos simples.
- El tiempo de entrenamiento crece linealmente con la cantidad de dimensiones.
- Rápido y simple de implementar.

Desventajas:

- La hipótesis de independencia no se cumple en muchos datos reales.

NB: pero no es tan *ingenuo*!

Ventajas:

- Puede usarse directamente en mezclas de datos continuos y categóricos.
- En varios conjuntos reales funciona mejor que los árboles de decisión y otros métodos simples.
- El tiempo de entrenamiento crece linealmente con la cantidad de dimensiones.
- Rápido y simple de implementar.

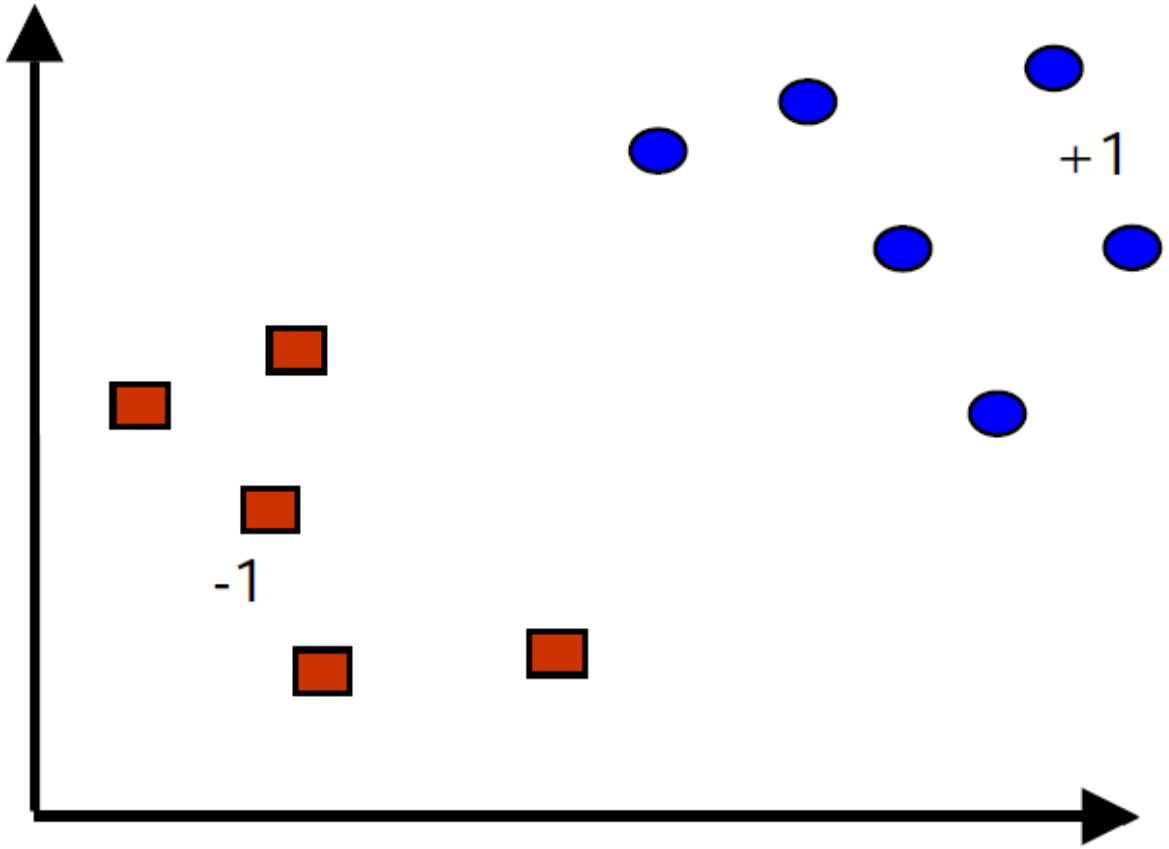
Desventajas:

- La hipótesis de independencia no se cumple en muchos datos reales.
- Suele presentar problemas de estimación durante el entrenamiento.

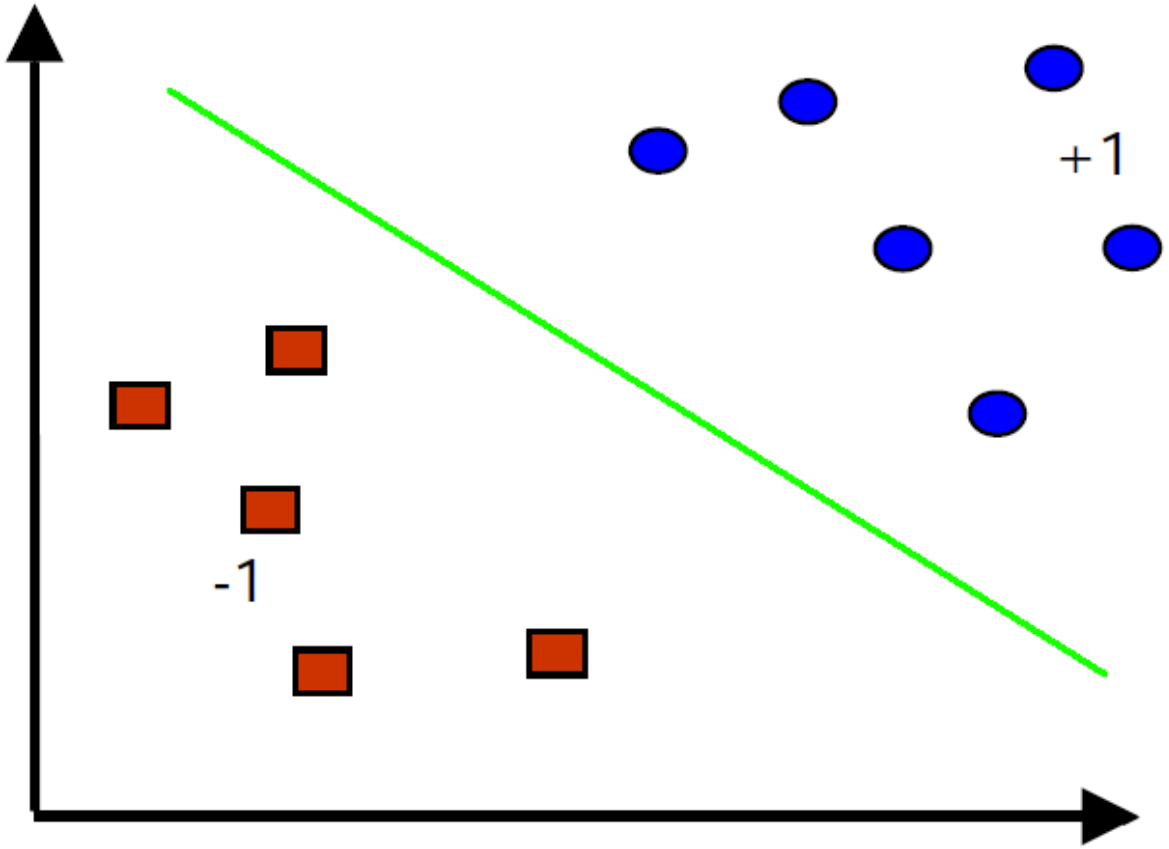
Máquinas con vectores de soporte

(SVM, del inglés *Support Vector Machines*)

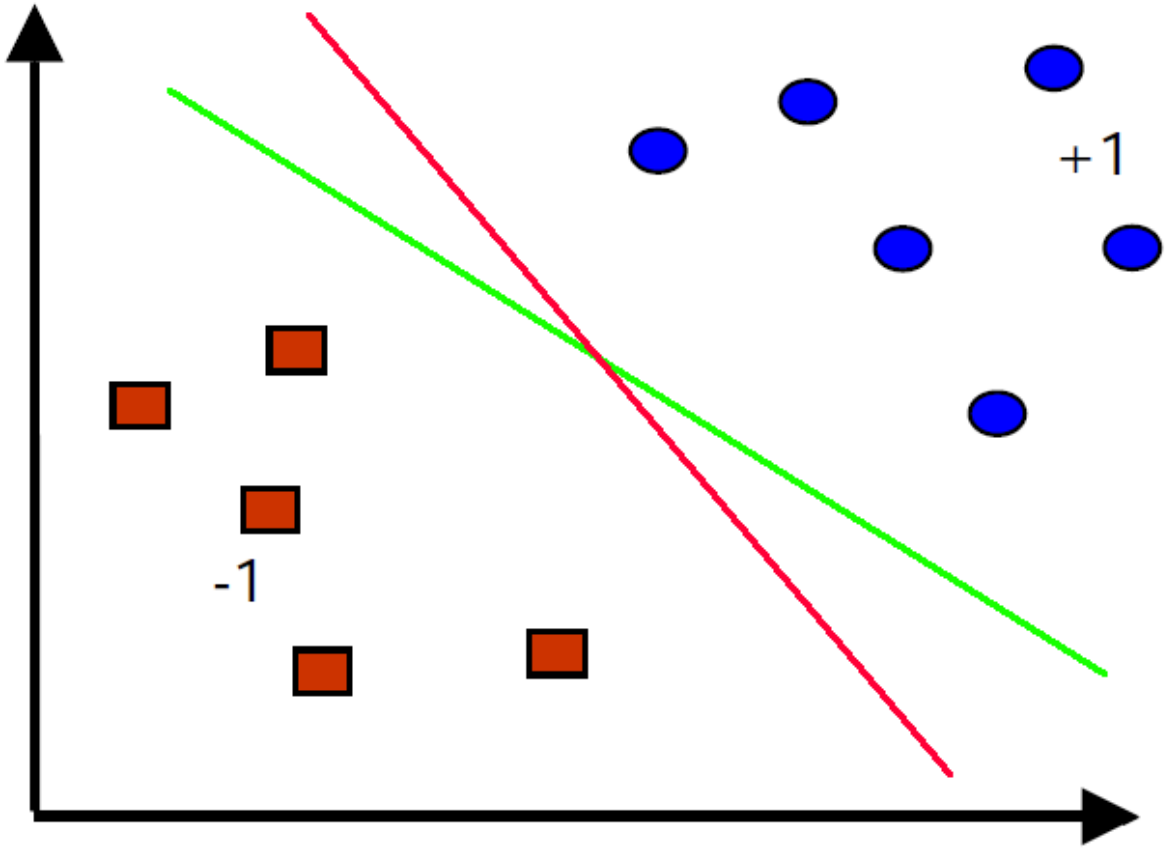
Margen: cuál sería el mejor clasificador?



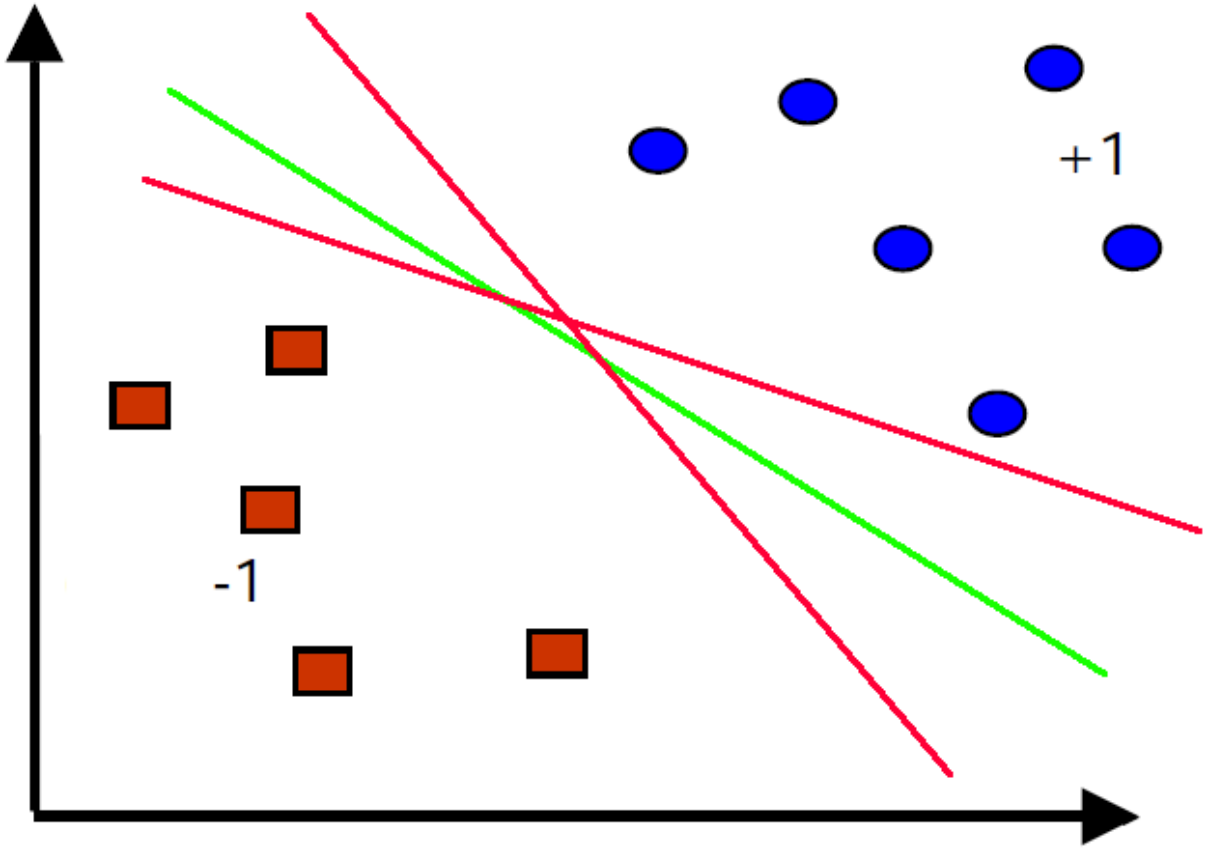
Margen: cuál sería el mejor clasificador?



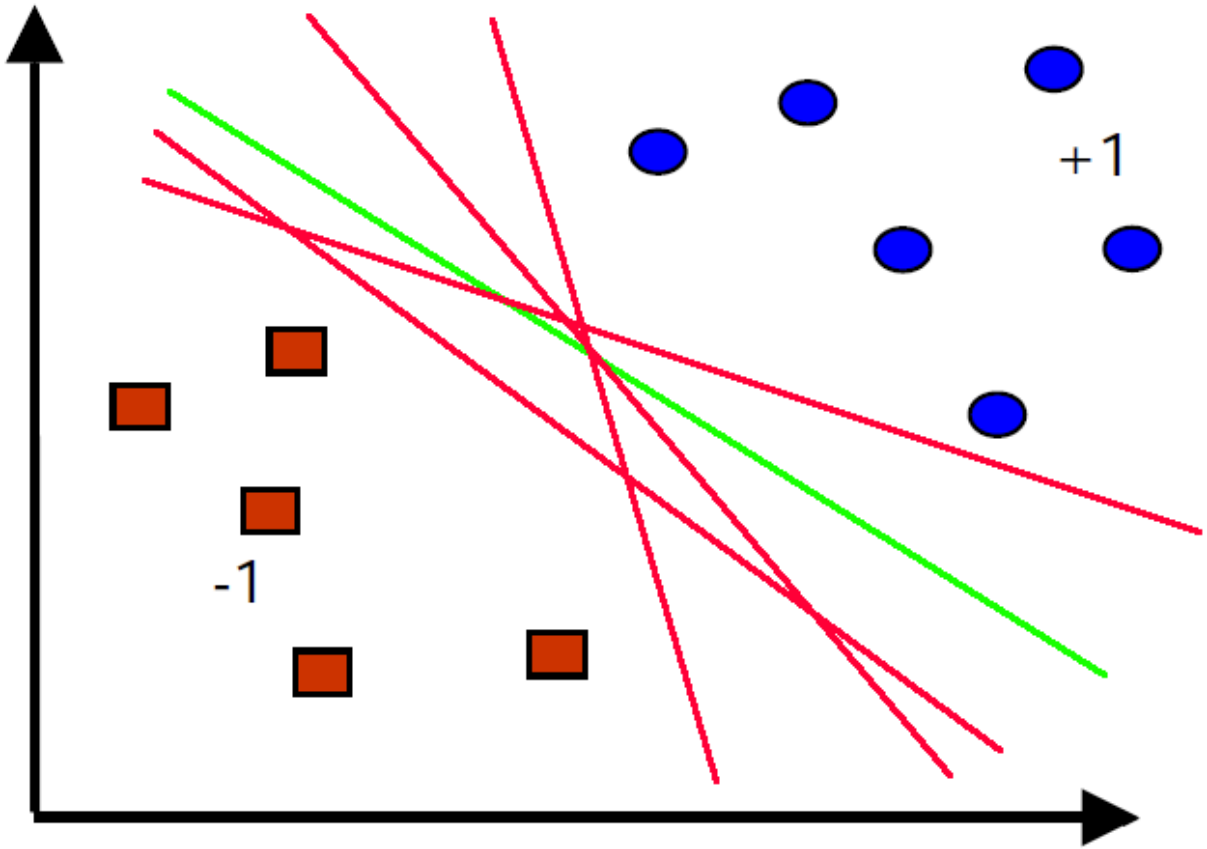
Margen: cuál sería el mejor clasificador?



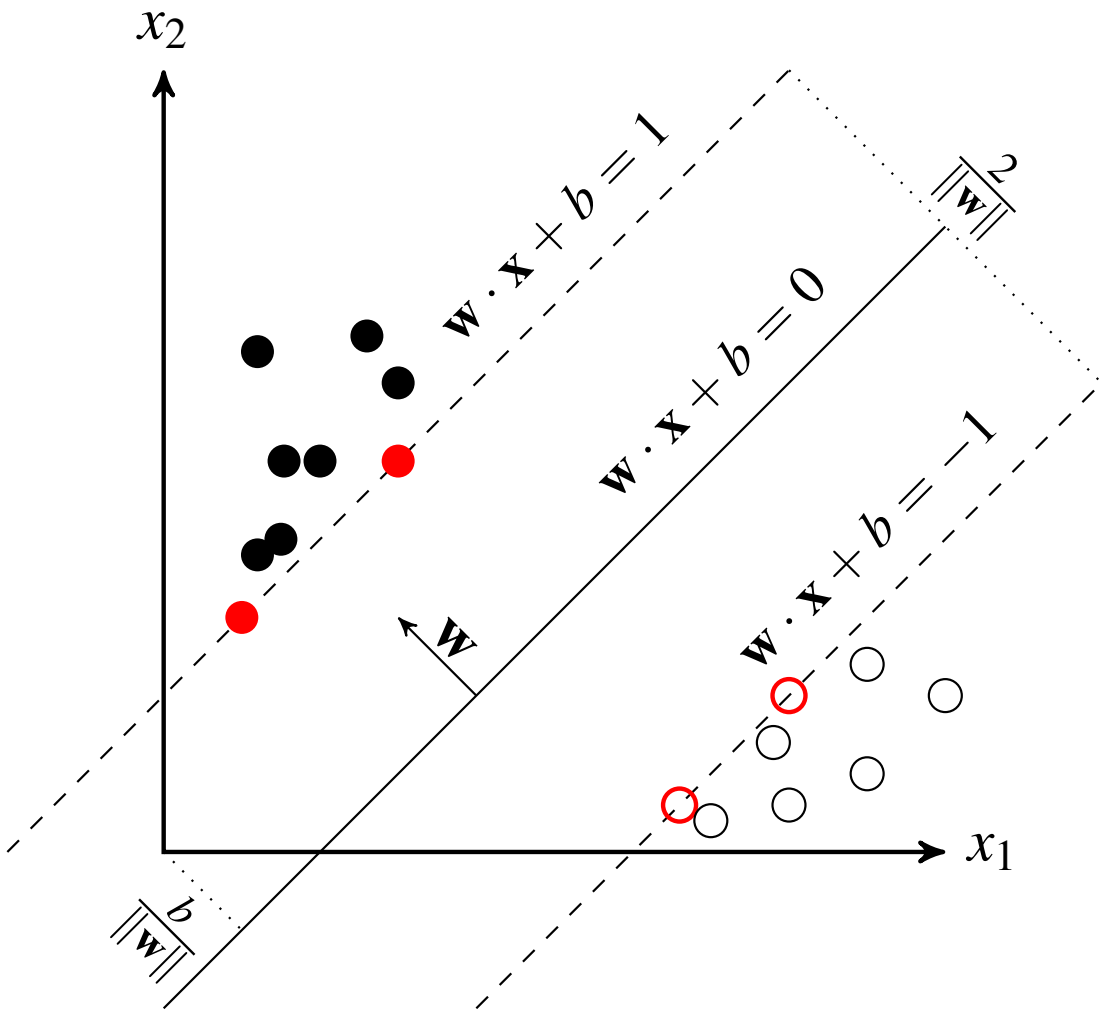
Margen: cuál sería el mejor clasificador?



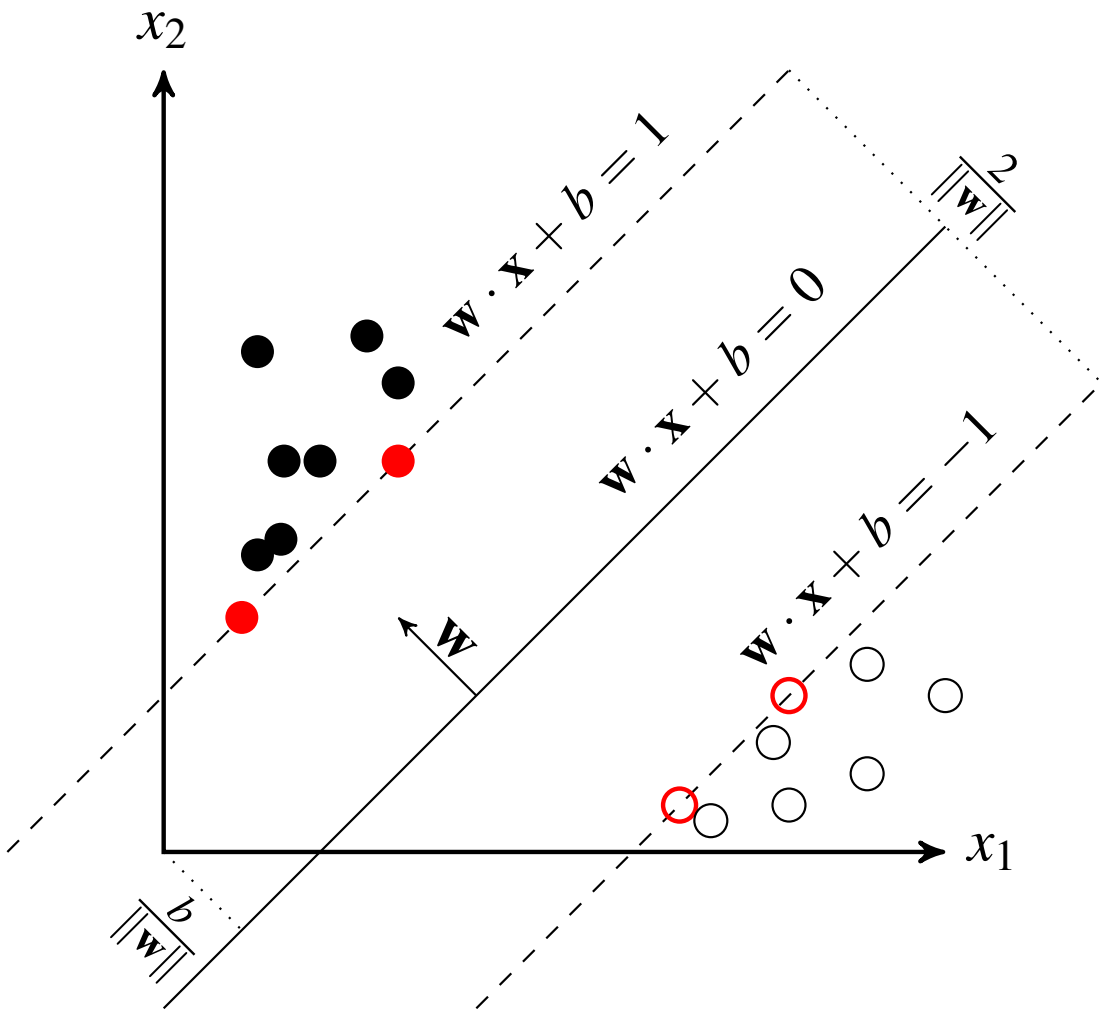
Margen: cuál sería el mejor clasificador?



SVM: maximizando el margen

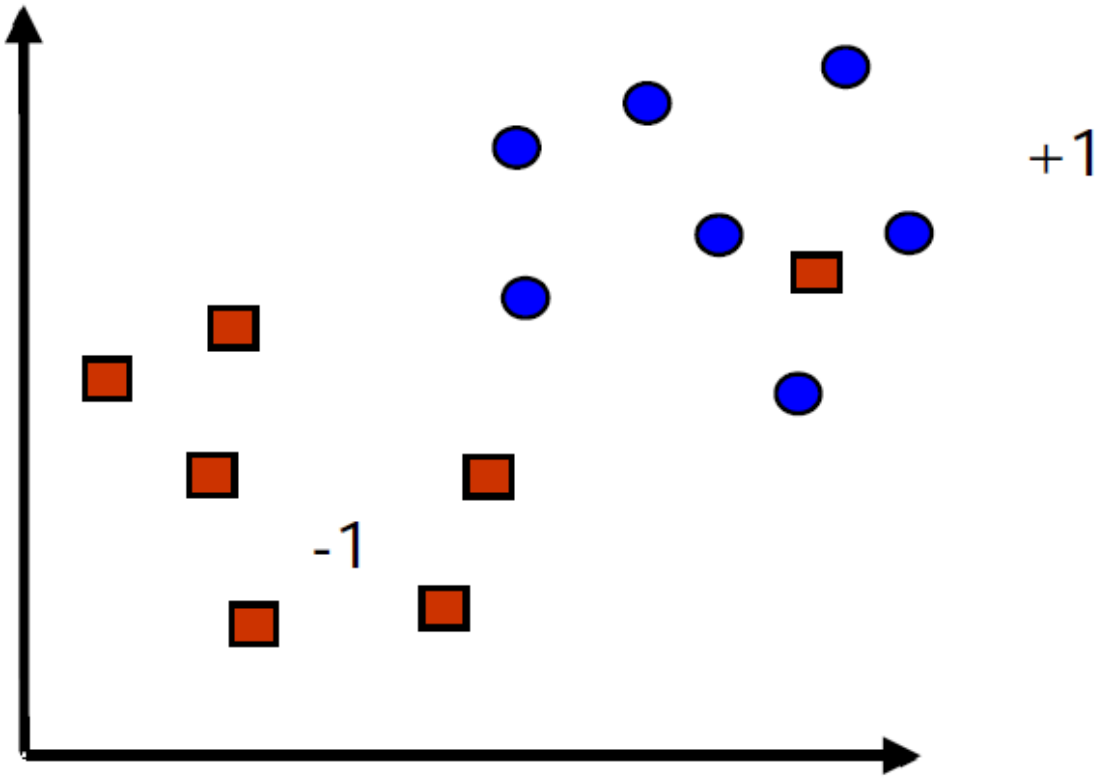


SVM: maximizando el margen

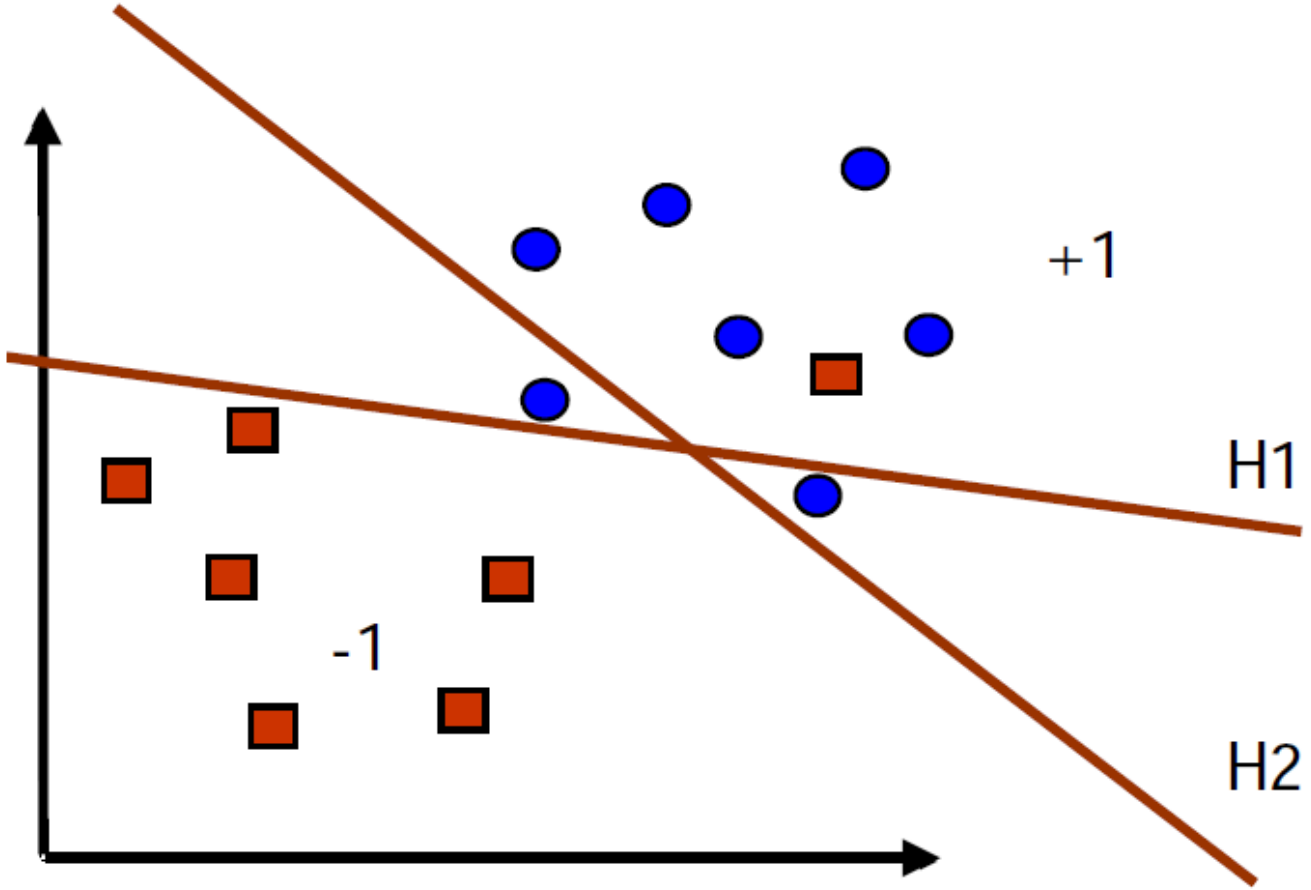


Vectores de soporte

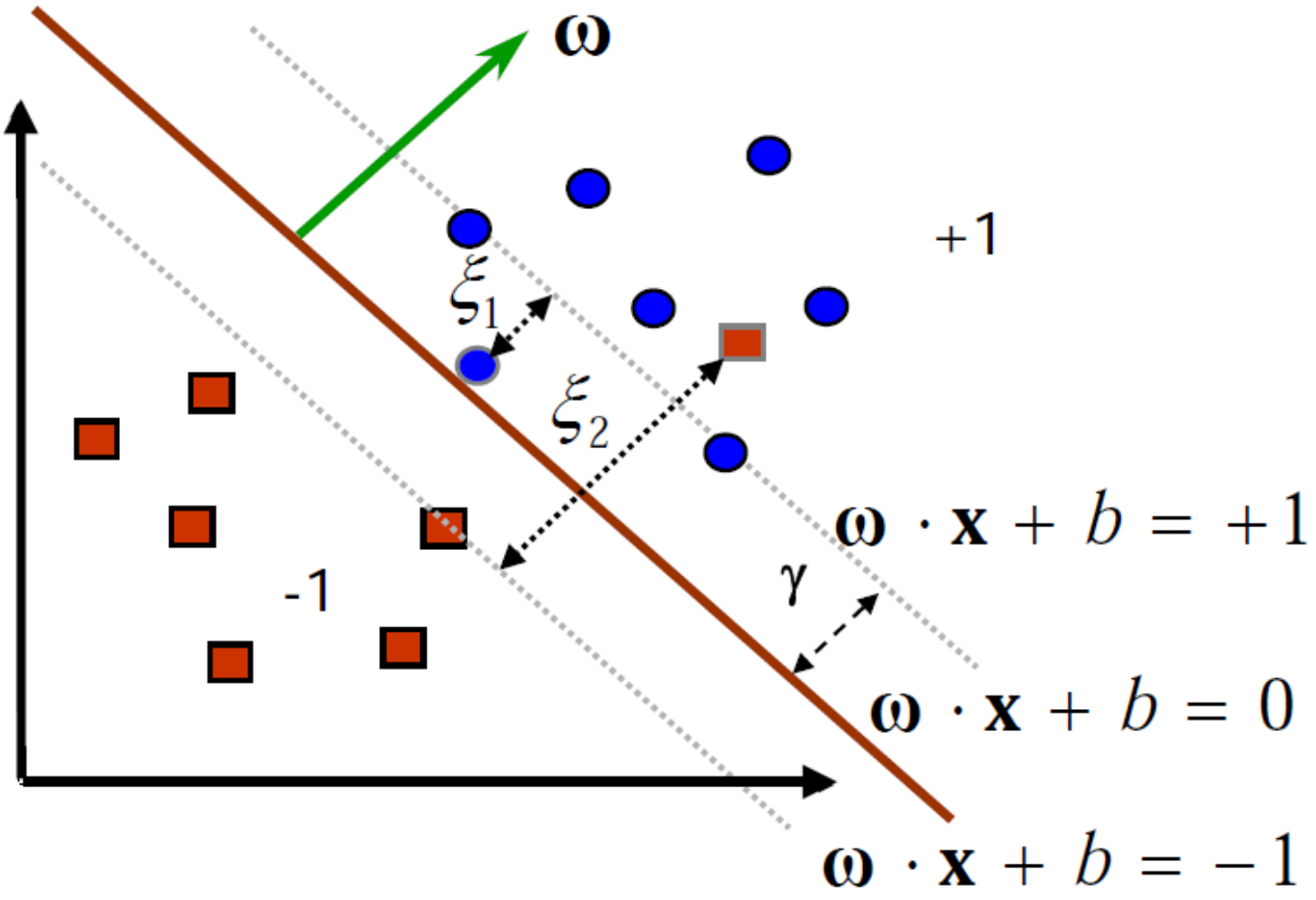
SVM: maximizando el margen



SVM: márgenes blandos

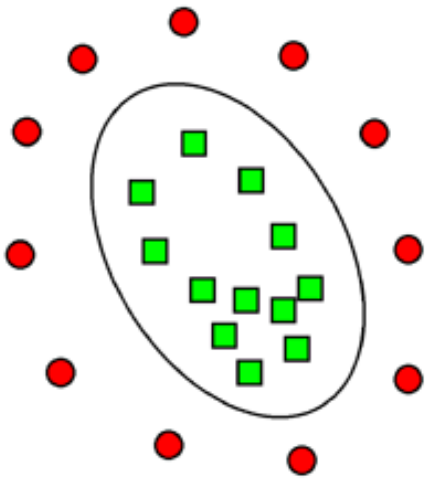


SVM: márgenes blandos

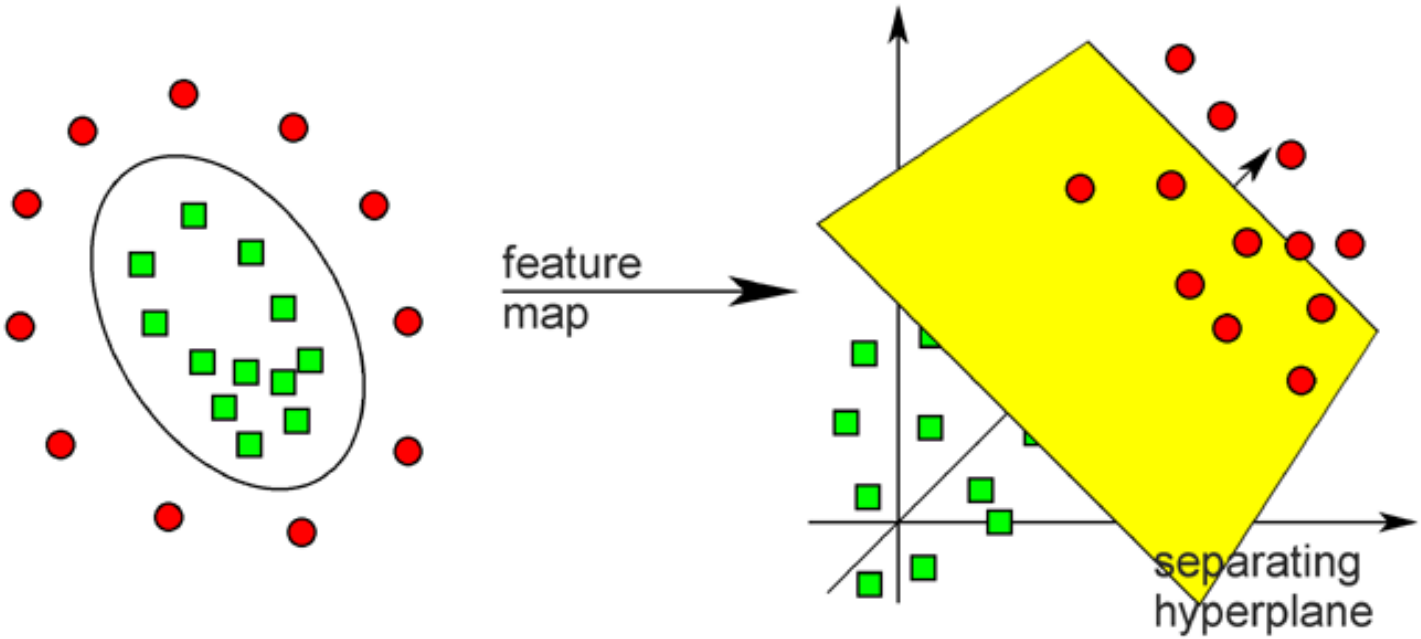


El truco del *kernel*

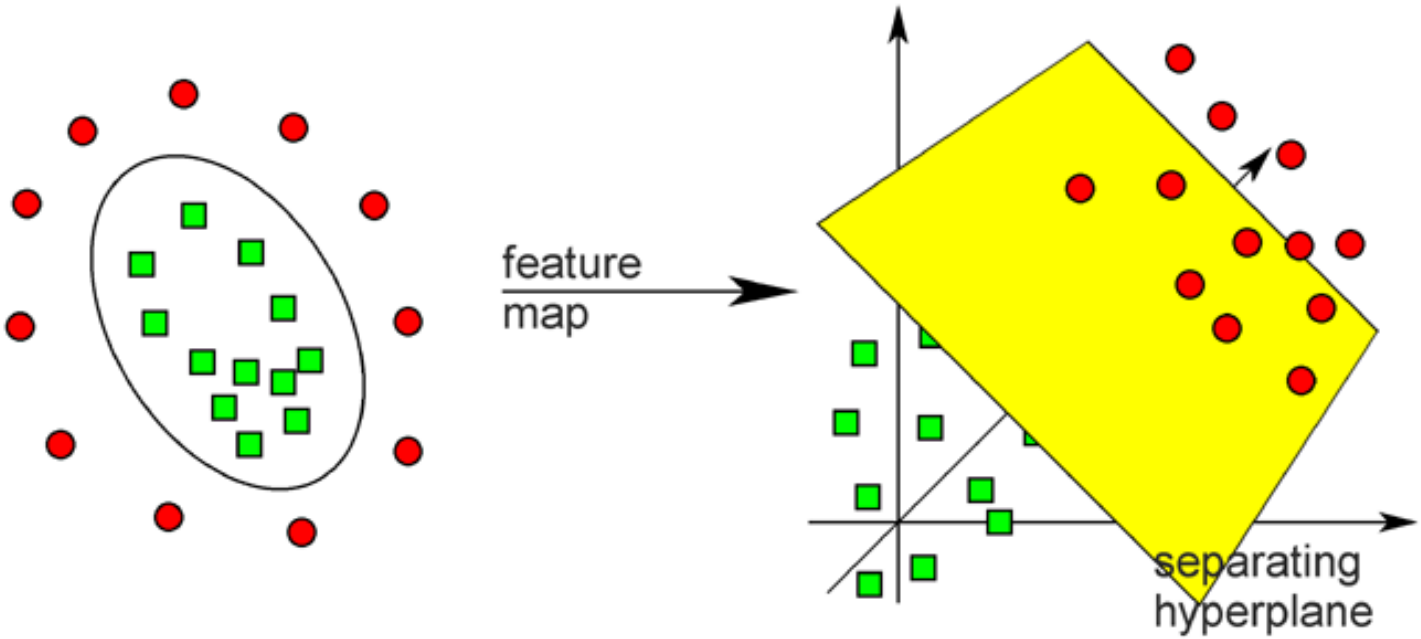
El truco del *kernel*



El truco del *kernel*



El truco del *kernel*



$$\phi : \mathbb{R}^2 \rightarrow \mathbb{R}^3$$

$$\phi : (x_1, x_2) \mapsto (z_1, z_2, z3) = (x_1^2, \sqrt{2}x_1x_2, x_2^2)$$

El truco del *kernel*

El truco del *kernel*

Algunos kernels:

- Gaussiano: $e^{-\left(\frac{\|\mathbf{x}-\mathbf{z}\|}{\sigma}\right)^2}$

El truco del *kernel*

Algunos kernels:

- Gaussiano: $e^{-\left(\frac{\|\mathbf{x}-\mathbf{z}\|}{\sigma}\right)^2}$
- Polinomial: $(\mathbf{x} \cdot \mathbf{z} + c)^d$

El truco del *kernel*

Algunos kernels:

- Gaussiano: $e^{-\left(\frac{\|\mathbf{x}-\mathbf{z}\|}{\sigma}\right)^2}$
- Polinomial: $(\mathbf{x} \cdot \mathbf{z} + c)^d$
- Sigmoideo: $\tanh\left(c(\mathbf{x} \cdot \mathbf{z}) + \theta\right)$

SVM

Ventajas:

- Rápida convergencia en entrenamiento.

Desventajas:

SVM

Ventajas:

- Rápida convergencia en entrenamiento.
- Funciona bien cuando los márgenes quedan naturalmente amplios en el espacio proyectado.

Desventajas:

SVM

Ventajas:

- Rápida convergencia en entrenamiento.
- Funciona bien cuando los márgenes quedan naturalmente amplios en el espacio proyectado.
- Funciona bastante bien en altas dimensiones de entrada.

Desventajas:

SVM

Ventajas:

- Rápida convergencia en entrenamiento.
- Funciona bien cuando los márgenes quedan naturalmente amplios en el espacio proyectado.
- Funciona bastante bien en altas dimensiones de entrada.
- Puede funcionar bien cuando la cantidad de dimensiones es mayor que la cantidad de datos de entrenamiento.

Desventajas:

SVM

Ventajas:

- Rápida convergencia en entrenamiento.
- Funciona bien cuando los márgenes quedan naturalmente amplios en el espacio proyectado.
- Funciona bastante bien en altas dimensiones de entrada.
- Puede funcionar bien cuando la cantidad de dimensiones es mayor que la cantidad de datos de entrenamiento.
- Suele proveer buena capacidad de generalización.

Desventajas:

SVM

Ventajas:

- Rápida convergencia en entrenamiento.
- Funciona bien cuando los márgenes quedan naturalmente amplios en el espacio proyectado.
- Funciona bastante bien en altas dimensiones de entrada.
- Puede funcionar bien cuando la cantidad de dimensiones es mayor que la cantidad de datos de entrenamiento.
- Suele proveer buena capacidad de generalización.

Desventajas:

- El desempeño es muy dependiente del kernel que se elija (y sus hiperparámetros).

SVM

Ventajas:

- Rápida convergencia en entrenamiento.
- Funciona bien cuando los márgenes quedan naturalmente amplios en el espacio proyectado.
- Funciona bastante bien en altas dimensiones de entrada.
- Puede funcionar bien cuando la cantidad de dimensiones es mayor que la cantidad de datos de entrenamiento.
- Suele proveer buena capacidad de generalización.

Desventajas:

- El desempeño es muy dependiente del kernel que se elija (y sus hiperparámetros).
- Tiene problemas con grandes conjuntos de datos.

SVM

Ventajas:

- Rápida convergencia en entrenamiento.
- Funciona bien cuando los márgenes quedan naturalmente amplios en el espacio proyectado.
- Funciona bastante bien en altas dimensiones de entrada.
- Puede funcionar bien cuando la cantidad de dimensiones es mayor que la cantidad de datos de entrenamiento.
- Suele proveer buena capacidad de generalización.

Desventajas:

- El desempeño es muy dependiente del kernel que se elija (y sus hiperparámetros).
- Tiene problemas con grandes conjuntos de datos.
- No maneja bien el solapamiento de clases.

SVM

Ventajas:

- Rápida convergencia en entrenamiento.
- Funciona bien cuando los márgenes quedan naturalmente amplios en el espacio proyectado.
- Funciona bastante bien en altas dimensiones de entrada.
- Puede funcionar bien cuando la cantidad de dimensiones es mayor que la cantidad de datos de entrenamiento.
- Suele proveer buena capacidad de generalización.

Desventajas:

- El desempeño es muy dependiente del kernel que se elija (y sus hiperparámetros).
- Tiene problemas con grandes conjuntos de datos.
- No maneja bien el solapamiento de clases.
- No provee buena interpretabilidad de las decisiones.

SVM

Ventajas:

- Rápida convergencia en entrenamiento.
- Funciona bien cuando los márgenes quedan naturalmente amplios en el espacio proyectado.
- Funciona bastante bien en altas dimensiones de entrada.
- Puede funcionar bien cuando la cantidad de dimensiones es mayor que la cantidad de datos de entrenamiento.
- Suele proveer buena capacidad de generalización.

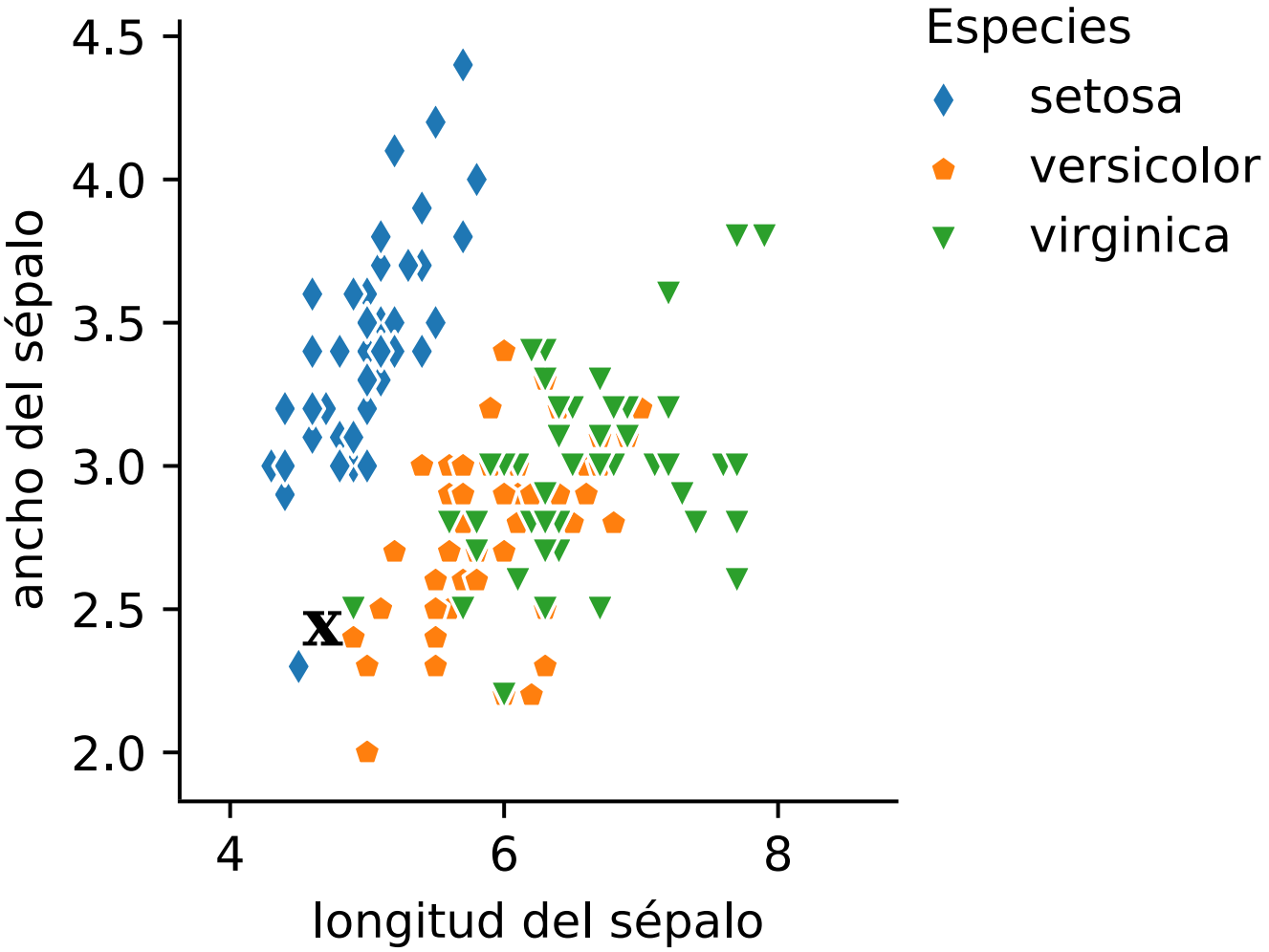
Desventajas:

- El desempeño es muy dependiente del kernel que se elija (y sus hiperparámetros).
- Tiene problemas con grandes conjuntos de datos.
- No maneja bien el solapamiento de clases.
- No provee buena interpretabilidad de las decisiones.
- No es directa la extensión a problemas multiclase.

Ensambles de clasificadores

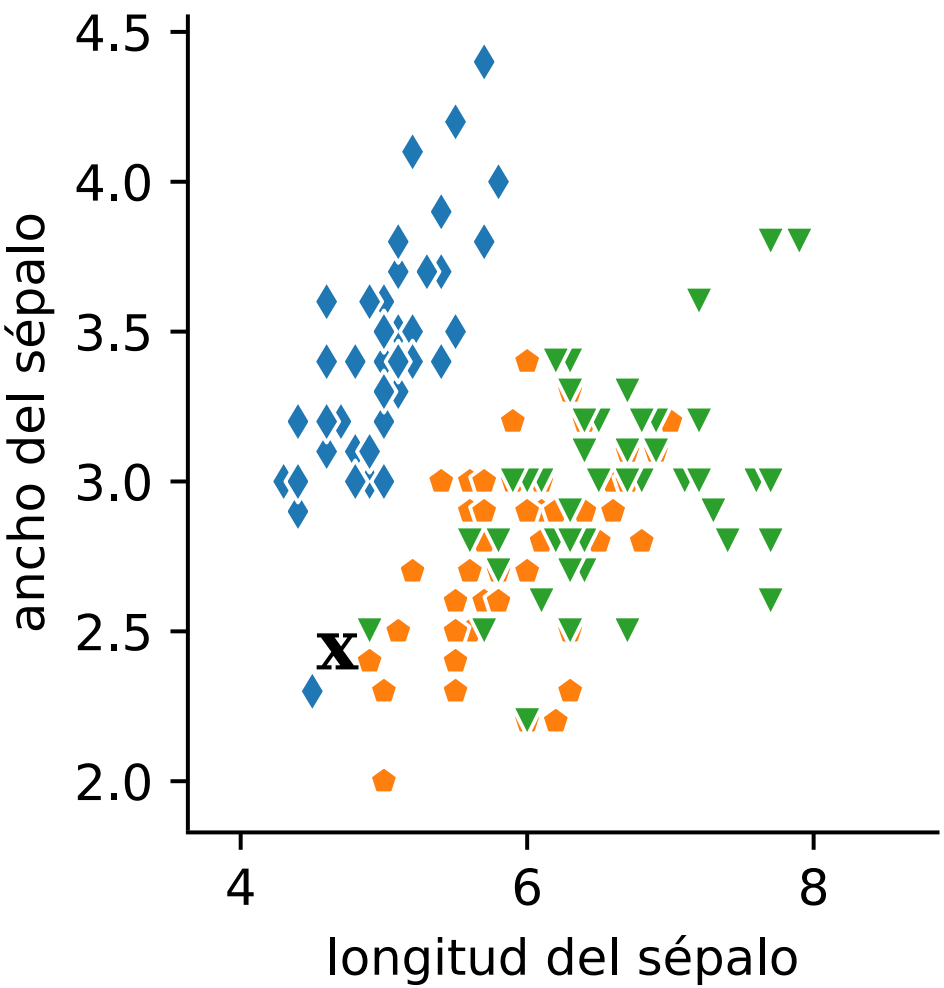
Ensambles de clasificadores

La idea: levanten la mano quienes piensan $\mathbf{x}$ que es versicolor!



Ensambles de clasificadores

La idea: levanten la mano quienes piensan **x** que es versicolor!



Combinar varios modelos simples en lugar de construir un modelo complejo.

Bagging

(del ingles *Bootstrap AGGregatING*)

Bagging

Entrenar varios modelos simples...

haciendo *bootstrap* sobre el conjunto de datos

Bagging

Entrenar varios modelos simples...
haciendo *bootstrap* sobre el conjunto de datos

Bagging:

1. Entrenar B modelos separados, cada uno para predecir $y_b(\mathbf{x})$.
2. La predicción del ensamble estará dada por simple votación de las salidas de cada modelo $y_b(\mathbf{x})$.
   En el caso de regresión es directamente el promedio $y_{ens}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} y_b(\mathbf{x})$

Bagging

Entrenar varios modelos simples...
haciendo *bootstrap* sobre el conjunto de datos

Bagging:

1. Entrenar B modelos separados, cada uno para predecir $y_b(\mathbf{x})$.
2. La predicción del ensamble estará dada por simple votación de las salidas de cada modelo $y_b(\mathbf{x})$.
   En el caso de regresión es directamente el promedio $y_{ens}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} y_b(\mathbf{x})$

Dados los errores individuales $\tilde{E}_b$, se puede demostrar que el error del ensamble es $\tilde{E}_E = \frac{1}{B}\tilde{E}_b$.

Bagging

Entrenar varios modelos simples...
haciendo *bootstrap* sobre el conjunto de datos

Bagging:

1. Entrenar B modelos separados, cada uno para predecir $y_b(\mathbf{x})$.
2. La predicción del ensamble estará dada por simple votación de las salidas de cada modelo $y_b(\mathbf{x})$.
   En el caso de regresión es directamente el promedio $y_{ens}(\mathbf{x}) = \frac{1}{B}\sum_{b=1}^{B} y_b(\mathbf{x})$

Dados los errores individuales $\tilde{E}_b$, se puede demostrar que el error del ensamble es $\tilde{E}_E = \frac{1}{B}\tilde{E}_b$.

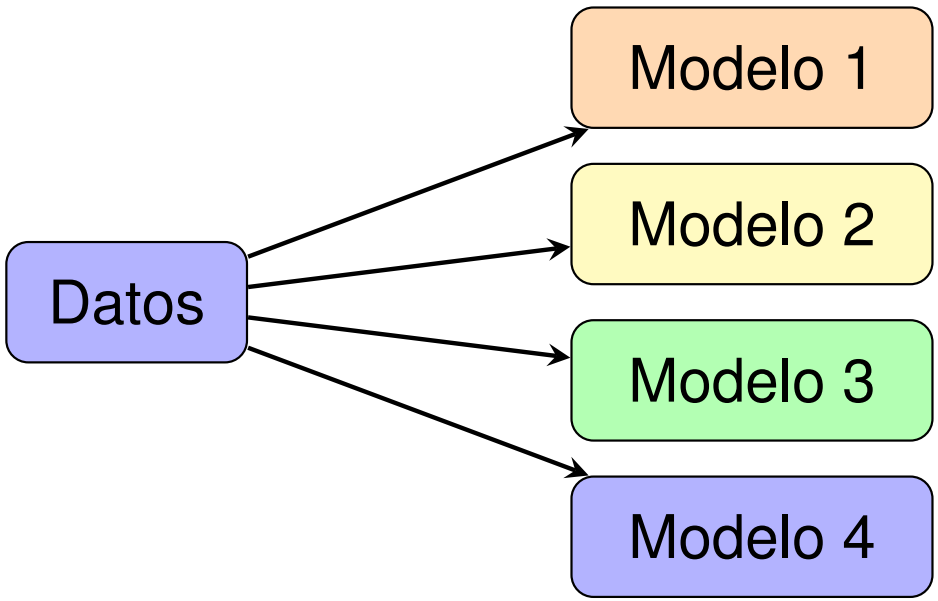
(Random Forests)

Stacking

Entrenar varios modelos simples... (como en bagging))
y luego un *meta*-modelo que recibe como entrada
las salidas de los modelos simples

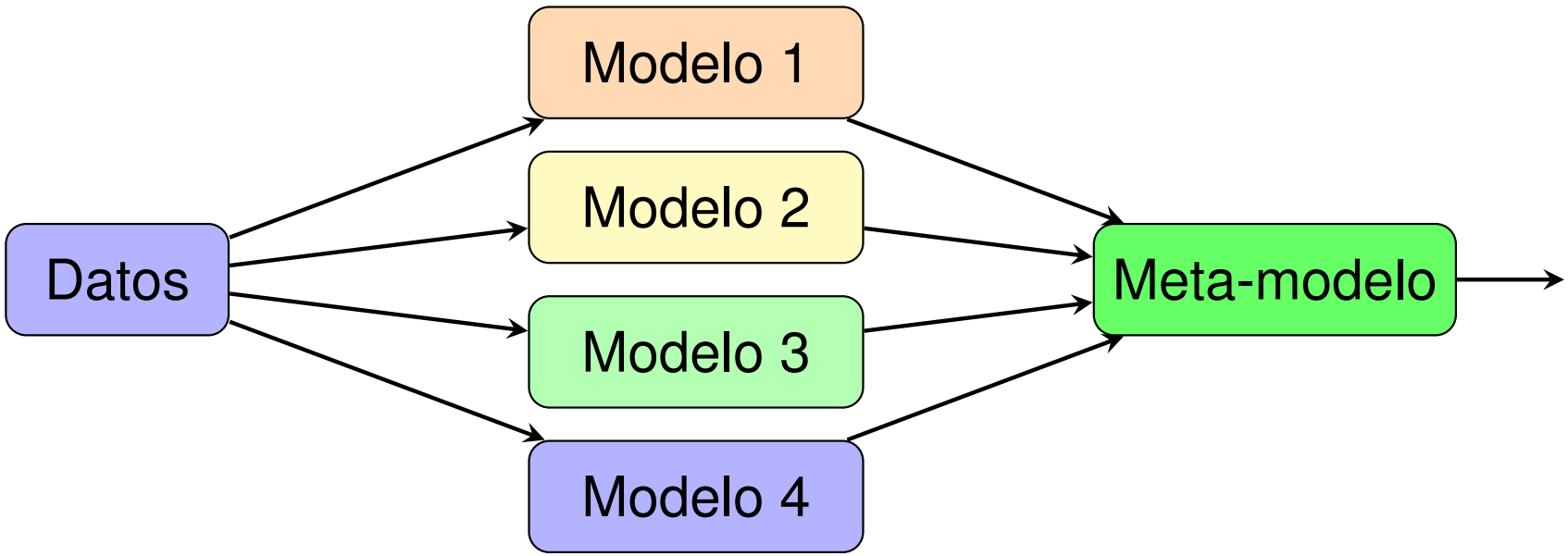
Stacking

Entrenar varios modelos simples... (como en bagging))
y luego un *meta*-modelo que recibe como entrada
las salidas de los modelos simples



Stacking

Entrenar varios modelos simples... (como en bagging))
y luego un *meta*-modelo que recibe como entrada
las salidas de los modelos simples



Stacking

Entrenar varios modelos simples... y luego un *meta*-modelo con las salidas de los modelos simples

Stacking:

1. Entrenar B modelos separados, cada uno para predecir $y_b(\mathbf{x})$.
2. Entrenar un meta-modelo $\mathcal{M}(\cdot)$ que recibe como entrada las $y_b(\mathbf{x})$ y predice la salida final

La predicción del ensamble estará la salida del meta modelo:

$$y_{ens}(\mathbf{x}) = \mathcal{M}\left(y_1(\mathbf{x}), y_2(\mathbf{x}), \ldots, y_B(\mathbf{x})\right)$$

Boosting

Boosting

Ideas clave:

- A diferencia del bagging, en boosting modificamos la distribución de los datos de entrenamiento antes de hacer un nuevo muestreo con bootstrap.

Boosting

Ideas clave:

- A diferencia del bagging, en boosting modificamos la distribución de los datos de entrenamiento antes de hacer un nuevo muestreo con bootstrap.
- Buscamos que la nueva muestra sea más informativa para el nuevo modelo a entrenar.

Boosting por muestreo 3

Dado un conjunto de entrenamiento $\mathcal{S}$ de N patrones, para un clasificador **binario**:

1. Se entrena el clasificador C_1 usando la muestra de bootstrap $\mathcal{S}_1$ de $N' < N$ patrones (sin reposición).
2. Se entrena el clasificador C_2 en un conjunto de entrenamiento $\mathcal{S}_2$ en el que **la mitad** de sus patrones hayan sido clasificados correctamente por C_1.
3. Se entrena el clasificador C_3 con los patrones en los que la predicción de C_1 y C_2 difieren.

Boosting por muestreo 3

Dado un conjunto de entrenamiento $\mathcal{S}$ de N patrones, para un clasificador **binario**:

1. Se entrena el clasificador C_1 usando la muestra de bootstrap $\mathcal{S}_1$ de $N' < N$ patrones (sin reposición).
2. Se entrena el clasificador C_2 en un conjunto de entrenamiento $\mathcal{S}_2$ en el que **la mitad** de sus patrones hayan sido clasificados correctamente por C_1.
3. Se entrena el clasificador C_3 con los patrones en los que la predicción de C_1 y C_2 difieren.

Para un dato de prueba: se usa la predicción de C_1 y C_2 si coinciden, y sino se usa la predicción de C_3 (equivalente a una votación).

AdaBoost

(del ingles *ADAptive BOOSTing*)

AdaBoost

La idea clave: prestar más atención a los patrones difíciles.

AdaBoost

La idea clave: prestar más atención a los patrones difíciles.

- Cada patrón de entrenamiento tiene asociado un *peso* que determina la probabilidad con la que va a ser elegido en el boosting.

AdaBoost

La idea clave: prestar más atención a los patrones difíciles.

- Cada patrón de entrenamiento tiene asociado un *peso* que determina la probabilidad con la que va a ser elegido en el boosting.
- Si el patrón se clasifica bien entonces la probabilidad de ser elegido nuevamente tiene que bajar, y a la inversa, aumentar si es mal clasificado.

AdaBoost

La idea clave: prestar más atención a los patrones difíciles.

- Cada patrón de entrenamiento tiene asociado un *peso* que determina la probabilidad con la que va a ser elegido en el boosting.
- Si el patrón se clasifica bien entonces la probabilidad de ser elegido nuevamente tiene que bajar, y a la inversa, aumentar si es mal clasificado.
- En test se usa un voto *pesado* de acuerdo al desempeño previo de cada clasificador del ensamble.

Algoritmo de AdaBoost

Algoritmo de AdaBoost

Entradas:

Conjunto de entrenamiento $\mathcal{S}$, de tamaño N,

con las etiquetas de referencia $y_i \in \Omega = \{\omega_1,\ \omega_2\}$.

Algoritmo de AdaBoost

Entradas:

Conjunto de entrenamiento $\mathcal{S}$, de tamaño N,

con las etiquetas de referencia $y_i \in \Omega = \{\omega_1,\ \omega_2\}$.

Modelo de clasificador simple (o débil).

B: cantidad de modelos.

Algoritmo de AdaBoost

Entrenamiento:

Inicialización: $D_1(i) = 1/N, \quad \forall i.$

para $b = 1, 2, \ldots, B$

1. Muestrear por bootstrap el subconjunto S_b^* de acuerdo a la distribución D_b.
2. Entrenar un clasificador débil con S_b^*, y obtener sus predicciones $h_b : X \to \Omega$
3. Calcular el error de las predicciones h_b:

$$\epsilon_b = \sum_i \delta(h_b(\mathbf{x}_i), y_i) D_b(i).$$

Algoritmo de AdaBoost

Entrenamiento:

Inicialización: $D_1(i) = 1/N, \quad \forall i.$

para $b = 1, 2, \ldots, B$

1. Muestrear por bootstrap el subconjunto S_b^* de acuerdo a la distribución D_b.
2. Entrenar un clasificador débil con S_b^*, y obtener sus predicciones $h_b : X \to \Omega$
3. Calcular el error de las predicciones h_b:

$$\epsilon_b = \sum_i \delta(h_b(\mathbf{x}_i), y_i) D_b(i).$$

4. **si** $\epsilon_b \leq \frac{1}{2} \Leftarrow$ tiene que ser **menor al "by chance"!**

Algoritmo de AdaBoost

Entrenamiento:

Inicialización: $D_1(i) = 1/N, \quad \forall i.$

para $b = 1, 2, \ldots, B$

1. Muestrear por bootstrap el subconjunto S_b^* de acuerdo a la distribución D_b.
2. Entrenar un clasificador débil con S_b^*, y obtener sus predicciones $h_b : X \to \Omega$
3. Calcular el error de las predicciones h_b:

$$\epsilon_b = \sum_i \delta(h_b(\mathbf{x}_i), y_i) D_b(i).$$

4. **si** $\epsilon_b \leq \frac{1}{2}$
   - 4.1 Calcular el error normalizado $\beta_b = \frac{\epsilon_b}{1-\epsilon_b}$
   - 4.2 Actualizar la distribución

$$D_{b+1}(i) = \frac{D_b(i)}{Z_b} \times \begin{cases} \beta_b, & \textbf{si } h_b(\mathbf{x}_i) = y_i \\ 1, & \text{en otro caso} \end{cases}$$

(Z_b: constante de normalización para las probabilidades).

Algoritmo AdaBoost

Prueba: Voto por mayoría pesado.

Algoritmo AdaBoost

Prueba: Voto por mayoría pesado.

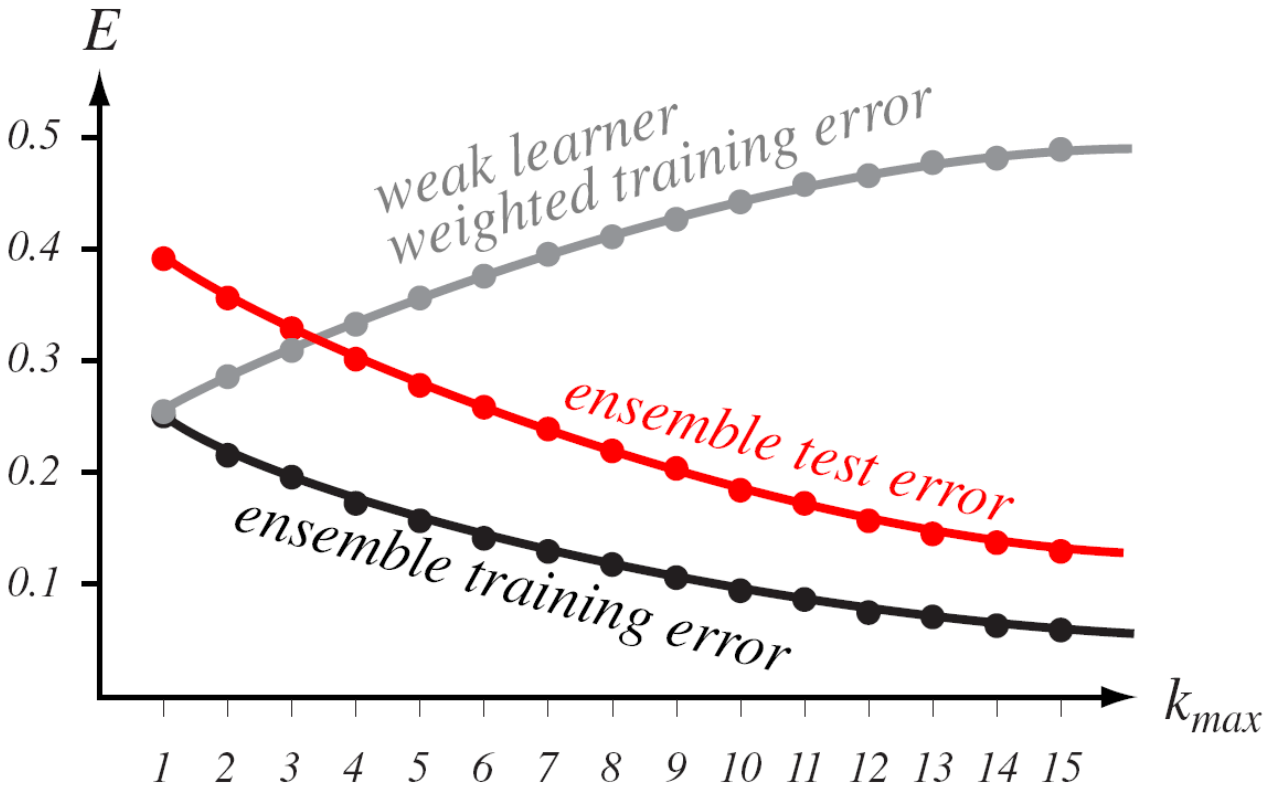
Dado un patrón no etiquetado $\mathbf{z}$

1. Obtener el total de votos recibido por cada clase

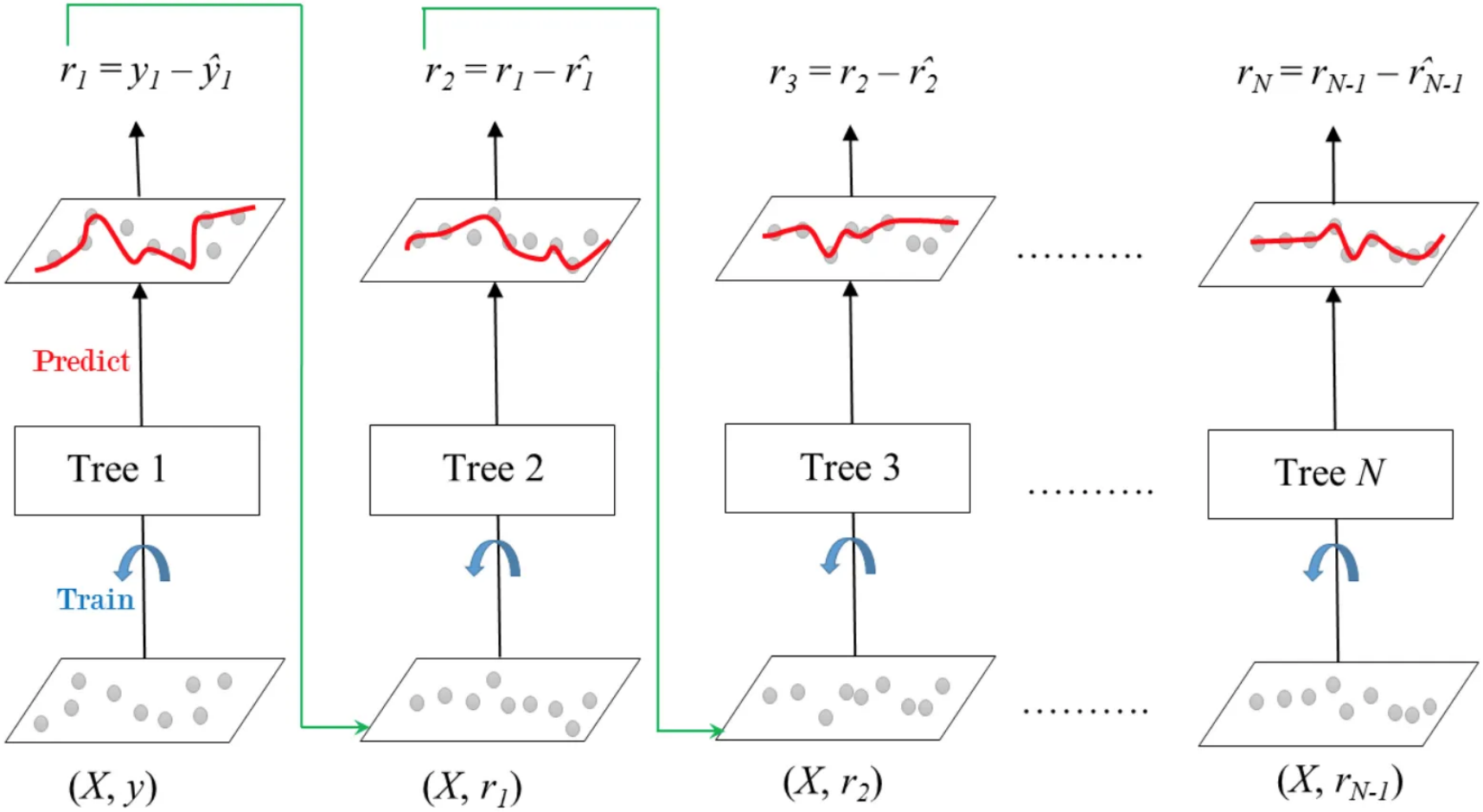
$$V_j = \sum_{b:h_b(\mathbf{z})=y_j} \log\left(1/\beta_b\right), \quad \forall j.$$

2. Elegir la clase que recibió más votos.

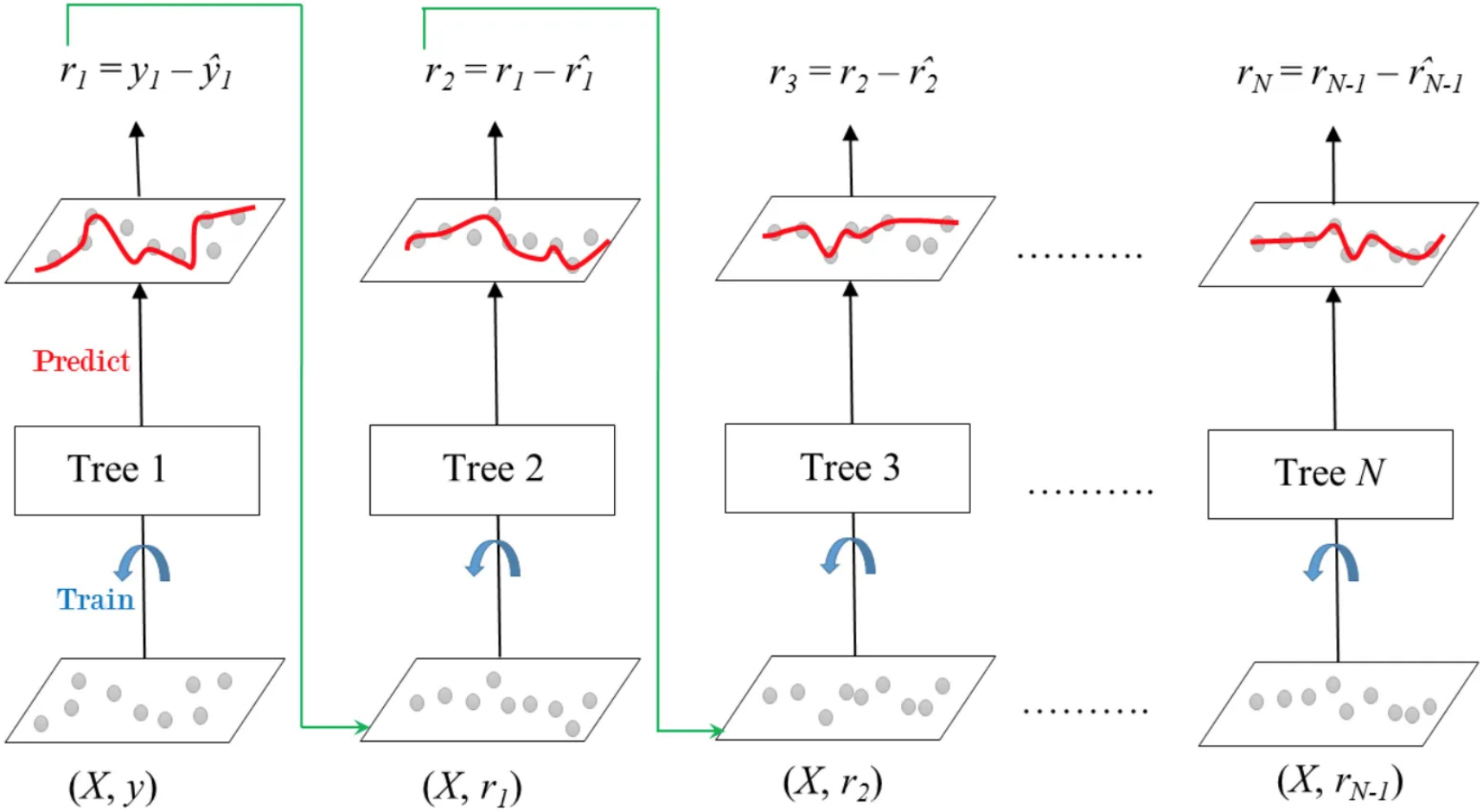
Convergencia de AdaBoost



Gradient boosting



Gradient boosting



Modelo final: $\hat{y} = \hat{y}_1 + \sum_i \eta \hat{r}_i$

Ensambles de clasificadores

Ventajas:

- Poco propensos al sobre-entrenamiento (buena capacidad de generalización).

Desventajas:

Ensambles de clasificadores

Ventajas:

- Poco propensos al sobre-entrenamiento (buena capacidad de generalización).
- Se adaptan al desbalance de clases.

Desventajas:

Ensambles de clasificadores

Ventajas:

- Poco propensos al sobre-entrenamiento (buena capacidad de generalización).
- Se adaptan al desbalance de clases.
- Pueden combinar ventajas de múltiples clasificadores y fuentes/tipos de datos.

Desventajas:

Ensambles de clasificadores

Ventajas:

- Poco propensos al sobre-entrenamiento (buena capacidad de generalización).
- Se adaptan al desbalance de clases.
- Pueden combinar ventajas de múltiples clasificadores y fuentes/tipos de datos.
- Hacen un mejor aprovechamiento de los datos cuando son escasos.

Desventajas:

Ensambles de clasificadores

Ventajas:

- Poco propensos al sobre-entrenamiento (buena capacidad de generalización).
- Se adaptan al desbalance de clases.
- Pueden combinar ventajas de múltiples clasificadores y fuentes/tipos de datos.
- Hacen un mejor aprovechamiento de los datos cuando son escasos.
- Tratan naturalmente los problemas multiclase.

Desventajas:

Ensambles de clasificadores

Ventajas:

- Poco propensos al sobre-entrenamiento (buena capacidad de generalización).
- Se adaptan al desbalance de clases.
- Pueden combinar ventajas de múltiples clasificadores y fuentes/tipos de datos.
- Hacen un mejor aprovechamiento de los datos cuando son escasos.
- Tratan naturalmente los problemas multiclase.
- Estables y robustos.

Desventajas:

Ensambles de clasificadores

Ventajas:

- Poco propensos al sobre-entrenamiento (buena capacidad de generalización).
- Se adaptan al desbalance de clases.
- Pueden combinar ventajas de múltiples clasificadores y fuentes/tipos de datos.
- Hacen un mejor aprovechamiento de los datos cuando son escasos.
- Tratan naturalmente los problemas multiclase.
- Estables y robustos.

Desventajas:

- El entrenamiento puede ser costoso.

Ensambles de clasificadores

Ventajas:

- Poco propensos al sobre-entrenamiento (buena capacidad de generalización).
- Se adaptan al desbalance de clases.
- Pueden combinar ventajas de múltiples clasificadores y fuentes/tipos de datos.
- Hacen un mejor aprovechamiento de los datos cuando son escasos.
- Tratan naturalmente los problemas multiclase.
- Estables y robustos.

Desventajas:

- El entrenamiento puede ser costoso.
- No son tan fáciles de interpretar.

Ensambles de clasificadores

Ventajas:

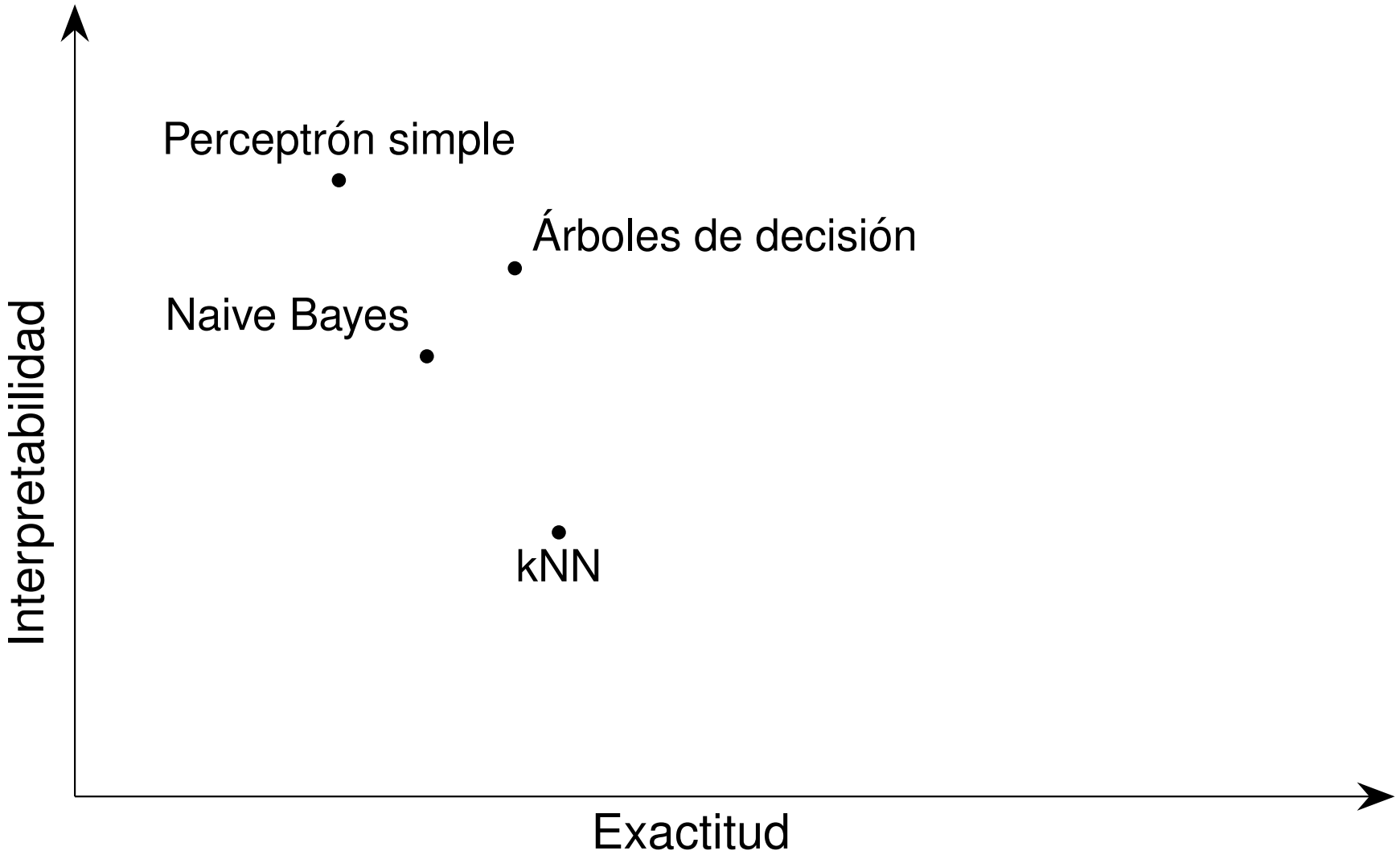
- Poco propensos al sobre-entrenamiento (buena capacidad de generalización).
- Se adaptan al desbalance de clases.
- Pueden combinar ventajas de múltiples clasificadores y fuentes/tipos de datos.
- Hacen un mejor aprovechamiento de los datos cuando son escasos.
- Tratan naturalmente los problemas multiclase.
- Estables y robustos.

Desventajas:

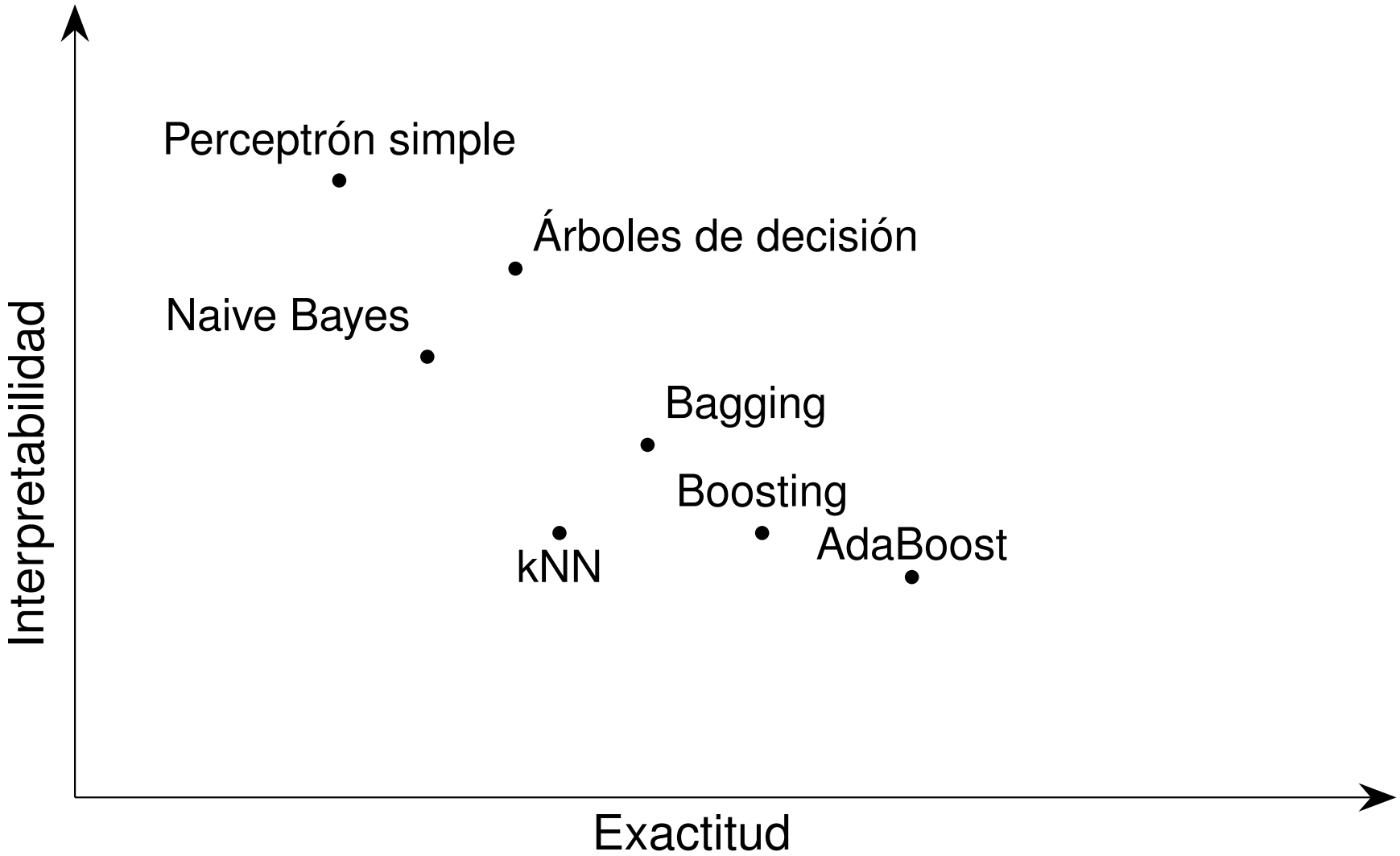
- El entrenamiento puede ser costoso.
- No son tan fáciles de interpretar.
- Los resultados pueden variar mucho dependiendo de la elección de los modelos débiles, su diversidad y la de los datos.

Interpretabilidad vs exactitud

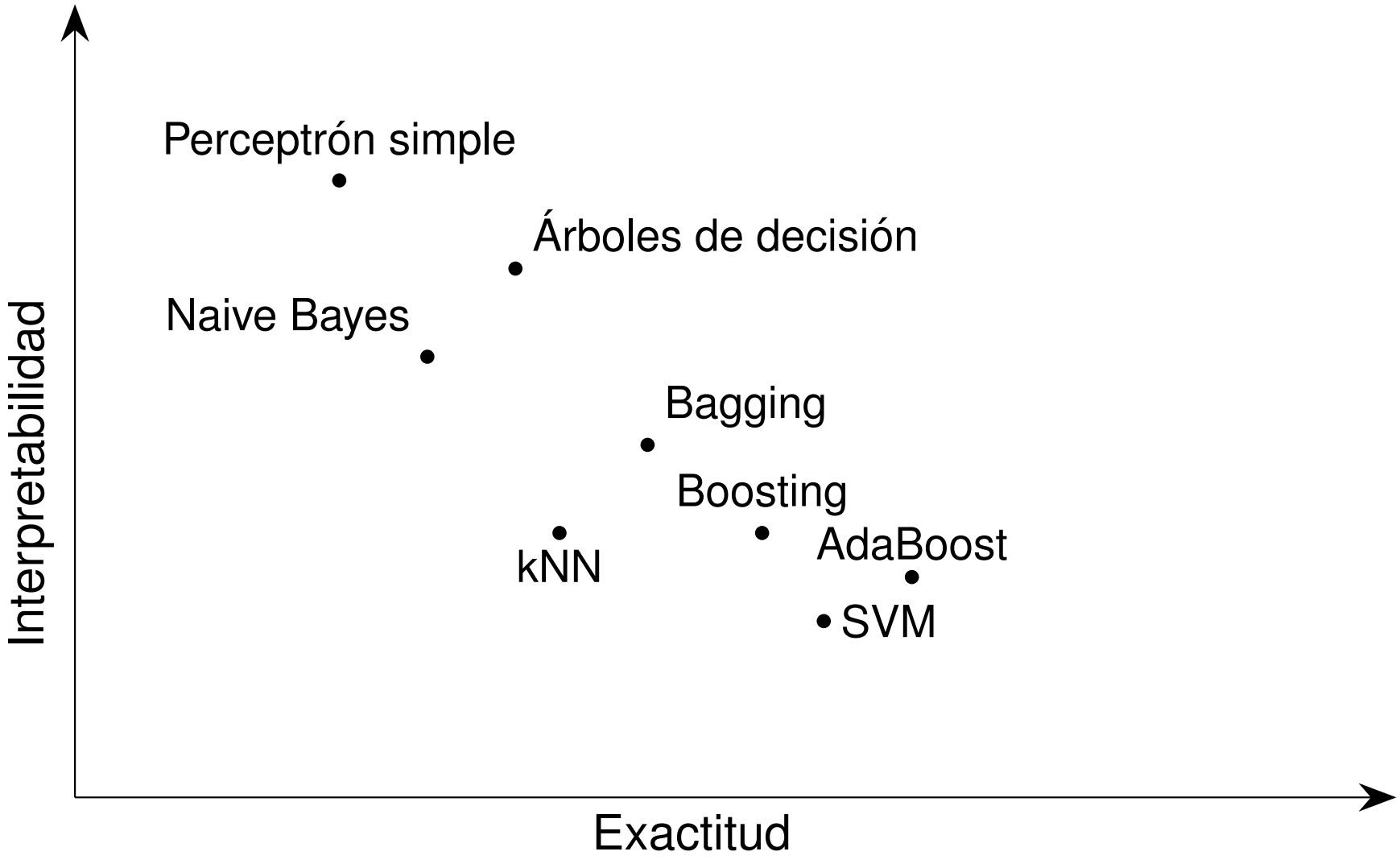
Interpretabilidad vs exactitud



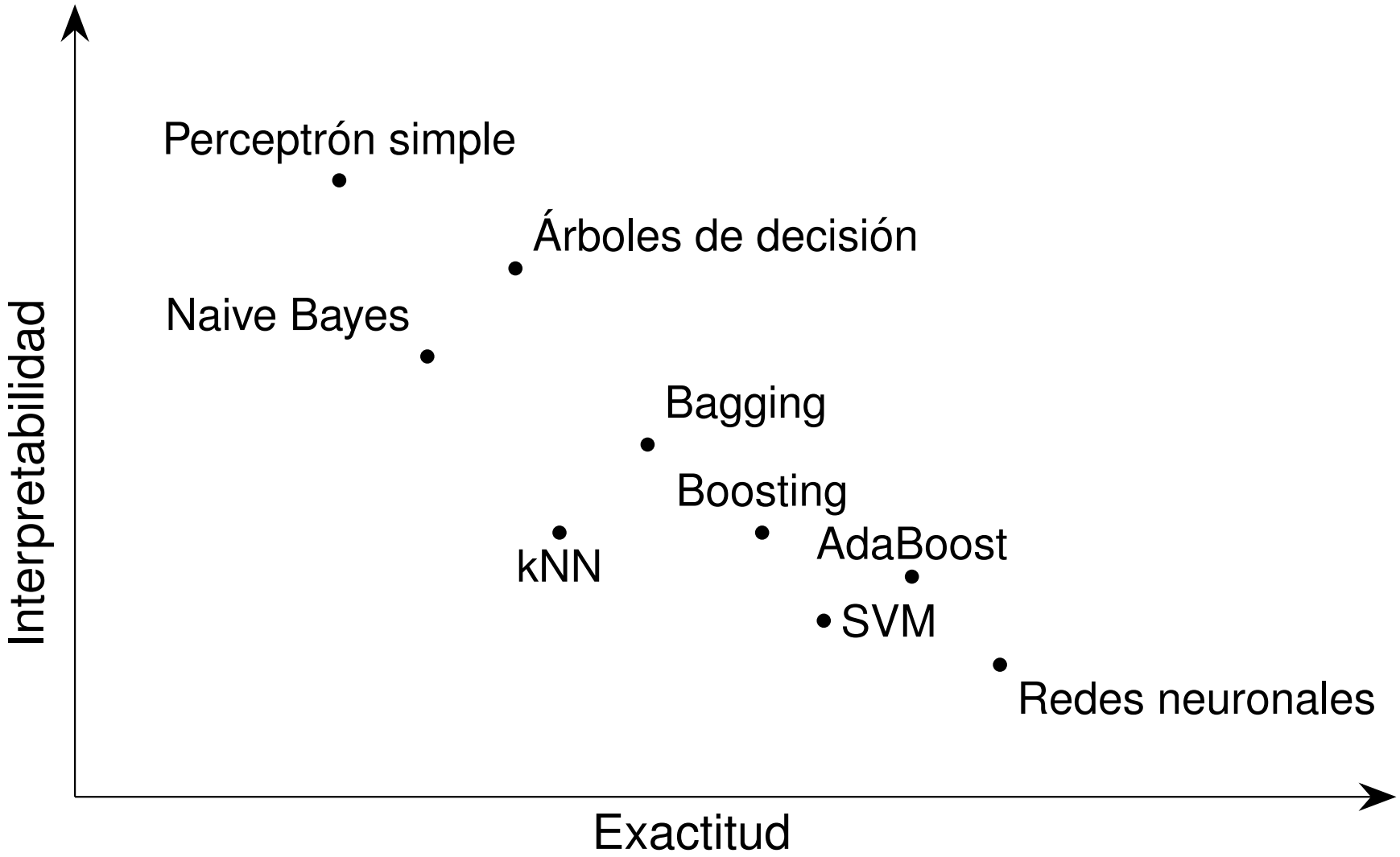
Interpretabilidad vs exactitud



Interpretabilidad vs exactitud



Interpretabilidad vs exactitud



Bibliografía

- C. Bishop, Pattern Recognition and Machine Learning, 2nd. Ed., 2006.
- R. Duda, P. Hart, D. Stork, Pattern Classification, 2nd. Ed., 2001.
- R. Polikar, Bootstrap-Inspired Techniques in Computacional Inteligence, IEEE Signal Processing Magazine, Vol. 24, No. 4, pp. 59-72, 2007.